

应用安全验证标准

5.0.0 版



2025 年 5 月

Contents

扉页 7

 关于本标准 7

 版权和许可 7

 项目负责人 7

 工作组 7

 其他主要贡献者 7

 其他贡献者和审阅者 8

前言 8

 引言 8

 5.0 版背后的原则 8

 展望未来 9

什么是 ASVS? 9

 ASVS 的范围 9

 应用 9

 安全 10

 验证 10

 标准 10

 要求 10

 已记录的安全决策 10

 应用安全验证等级 11

 等级评估 11

 第 1 级 11

 第 2 级 12

 第 3 级 12

 应达到哪个等级 12

 如何使用 ASVS 12

 ASVS 的结构 12

 发布策略 13

 灵活使用 ASVS 13

 如何引用 ASVS 要求 13

 ASVS 分支 14

 ASVS 的使用场景 14

 作为详细的安全架构指南 14

 作为专门的安全编码参考 14

 作为自动化单元测试和集成测试指南 14

 用于安全开发培训 14

 作为安全软件采购指导框架 15

 在实践中应用 ASVS 15

评估和认证	15
OWASP 对 ASVS 认证和信任标志的立场	15
如何验证 ASVS 合规性	15
验证报告	15
验证范围	15
验证机制	16
与 v4.x 相比的变化	16
引言	16
要求理念	17
范围和重点	17
更强调安全目标，而不是具体机制	17
已记录的安全决策	17
结构变化和新增章节	17
移除与其他标准的直接映射	18
降低与 NIST 数字身份指南的耦合	18
不再直接映射通用弱点枚举（Common Weakness Enumeration, CWE）	18
重新思考等级定义	18
更容易入门的等级	18
可测试性谬误	19
不只是风险	19
V1 编码和净化	19
控制目标	19
V1.1 编码和净化架构	19
V1.2 注入预防	20
V1.3 净化	21
V1.4 内存、字符串和非托管代码	22
V1.5 安全反序列化	22
参考资料	23
V2 验证和业务逻辑	23
控制目标	23
V2.1 验证和业务逻辑文档	24
V2.2 输入验证	24
V2.3 业务逻辑安全	25
V2.4 反自动化	25
参考资料	25
V3 Web 前端安全	26
控制目标	26
V3.1 Web 前端安全文档	26

V3.2 非预期内容解释	26
V3.3 Cookie 设置	27
V3.4 浏览器安全机制头	27
V3.5 浏览器源隔离	28
V3.6 外部资源完整性	29
V3.7 其他浏览器安全考虑	29
参考资料	30
V4 API 和 Web 服务	30
控制目标	30
V4.1 通用 Web 服务安全	30
V4.2 HTTP 消息结构验证	31
V4.3 GraphQL	31
V4.4 WebSocket	32
参考资料	32
V5 文件处理	32
控制目标	32
V5.1 文件处理文档	32
V5.2 文件上传和内容	33
V5.3 文件存储	33
V5.4 文件下载	34
参考资料	34
V6 认证	34
控制目标	34
V6.1 认证文档	35
V6.2 密码安全	35
V6.3 通用认证安全	36
V6.4 认证因素生命周期和恢复	37
V6.5 通用多因素认证要求	37
V6.6 带外认证机制	38
V6.7 密码学认证机制	39
V6.8 使用身份提供方进行认证	39
参考资料	40
V7 会话管理	40
控制目标	40
V7.1 会话管理文档	41
V7.2 基础会话管理安全	41
V7.3 会话超时	41
V7.4 会话终止	42

V7.5 防范会话滥用	42
V7.6 联合重新认证	43
参考资料	43
V8 授权	43
控制目标	43
V8.1 授权文档	44
V8.2 通用授权设计	44
V8.3 操作级授权	44
V8.4 其他授权考虑	45
参考资料	45
V9 自包含令牌	45
控制目标	45
V9.1 令牌来源和完整性	46
V9.2 令牌内容	46
参考资料	47
V10 OAuth 和 OIDC	47
控制目标	47
V10.1 通用 OAuth 和 OIDC 安全	48
V10.2 OAuth 客户端	48
V10.3 OAuth 资源服务器	49
V10.4 OAuth 授权服务器	49
V10.5 OIDC 客户端	51
V10.6 OpenID Provider	51
V10.7 同意管理	52
参考资料	52
V11 密码学	53
控制目标	53
V11.1 密码资产清单和文档	53
V11.2 密码技术的安全实现	54
V11.3 加密算法	54
V11.4 哈希和基于哈希的函数	55
V11.5 随机值	55
V11.6 公钥密码学	56
V11.7 使用中数据的密码保护	56
参考资料	56
V12 安全通信	57
控制目标	57
V12.1 通用 TLS 安全指导	57

V12.2 与面向外部的服务进行 HTTPS 通信	57
V12.3 通用服务间通信安全	58
参考资料	58
V13 配置	59
控制目标	59
V13.1 配置相关文档	59
V13.2 后端通信配置	59
V13.3 秘密管理	60
V13.4 非预期信息泄露	60
参考资料	61
V14 数据保护	61
控制目标	61
V14.1 数据保护文档	61
V14.2 通用数据保护	62
V14.3 客户端数据保护	62
参考资料	63
V15 安全编码和架构	63
控制目标	63
V15.1 安全编码和架构文档	63
V15.2 安全架构和依赖	64
V15.3 防御式编码	65
V15.4 安全并发	65
参考资料	66
V16 安全日志和错误处理	66
控制目标	66
V16.1 安全日志文档	66
V16.2 通用日志记录	67
V16.3 安全事件	67
V16.4 日志保护	68
V16.5 错误处理	68
参考资料	69
V17 WebRTC	69
控制目标	69
V17.1 TURN 服务器	69
V17.2 媒体	70
V17.3 信令	71
参考资料	71

附录 A：术语表	71
附录 B：参考资料	75
OWASP 核心项目	75
OWASP Cheat Sheet Series 项目	75
移动安全相关项目	75
OWASP 物联网相关项目	75
OWASP Serverless 项目	76
其他	76
附录 C：密码学标准	76
密码资产清单和文档	76
密码学参数的等效安全强度	77
随机值	77
加密算法	78
AES 加密模式	79
密钥封装	79
认证加密	80
哈希函数	81
通用场景下的哈希函数	81
用于密码存储的哈希函数	83
密钥派生函数（KDF）	84
通用密钥派生函数	84
基于密码的密钥派生函数	84
密钥交换机制	85
密钥交换方案	85
Diffie-Hellman 群	86
消息认证码（MAC）	87
数字签名	87
后量子加密标准	88
附录 D：建议	88
引言	88
建议采用、且属于范围内的机制	88
软件安全原则	89
软件安全流程	89
附录 E：贡献者	89

扉页

关于本标准

应用安全验证标准（Application Security Verification Standard, ASVS）收录了一组应用安全要求，可供架构师、开发人员、测试人员、安全专业人员、工具厂商和使用方用于定义、构建、测试和验证安全的应用。

版权和许可

版本 5.0.0，2025 年 5 月



Figure 1: license

版权所有 © 2008-2025 The OWASP Foundation。

本文档依据 Creative Commons Attribution-ShareAlike 4.0 International License 发布。

复用或分发本作品时，必须向接收方明确说明本作品所采用的许可条款。

项目负责人

Elar Lang	Josh C Grossman
Jim Manico	Daniel Cuthbert

工作组

Tobias Ahnoff	Ralph Andalis	Ryan Armstrong	Gabriel Corona
Meghan Jacquot	Shanni Prutchi	Iman Sharafaldin	Eden Yardeni

其他主要贡献者

Sjoerd Langkemper	Isaac Lewis
Mark Carney	Sandro Gauci

其他贡献者和审阅者

我们已在附录 E 中列出其他贡献者。

如果 5.x 贡献者名单中存在署名遗漏，请在 GitHub 上提交工单，以便在后续 5.x 更新中补充。

应用安全验证标准建立在 ASVS 1.0（2008）至 4.0（2019）历届参与者工作的基础之上。ASVS 沿用至今的许多结构和验证项，最初由 Andrew van der Stock、Mike Boberski、Jeff Williams、Dave Wichers 等众多贡献者编写。谨向所有曾参与本项目的人致谢。早期贡献者的完整名单请参阅各个历史版本。

前言

欢迎阅读应用安全验证标准（ASVS）5.0版。

引言

ASVS最初于 2008 年由全球社区协作推出，它为现代 Web 应用和服务的设计、开发与测试定义了一套完整的安全要求。

继 2019 年发布 ASVS 4.0、2021 年发布小版本更新 v4.0.3 之后，5.0 版成为又一个重要里程碑。本次修订紧跟软件安全领域的最新发展，对标准进行了全面更新。

ASVS 5.0 汇集了项目负责人、工作组成员及广大 OWASP 社区成员的共同努力，使这项重要标准得到了进一步更新和完善。

5.0 版背后的原则

本次重大修订遵循了几项关键原则：

- 范围和重点更清晰：本版标准更直接地围绕其名称中的四个核心支柱展开：应用、安全、验证和标准。要求文本经过重写，重点强调预防安全缺陷，而不是强制指定某种技术实现。要求文本应清楚说明自身含义及设立原因。
- 支持记录安全决策：ASVS 5.0 引入了记录关键安全决策的要求。这有助于增强可追溯性，并支持结合具体情境实施安全措施，使组织能够根据自身需求和风险调整安全态势。
- 更新分级：ASVS 仍保留三级模型，但调整了各等级的定义，使其更便于落地。第 1 级是采用 ASVS 的起点，提供第一层防护；第 2 级全面涵盖标准安全实践；第 3 级则涵盖高级别、高保证的要求。
- 内容重组和扩展：ASVS 5.0 在 17 个章节中包含约 350 条要求。各章节经过重新组织，以提升清晰度和易用性。同时提供了 v4.0 与 v5.0 之间的双向映射，便于迁移。

展望未来

应用安全建设永无止境，ASVS也会不断发展。5.0 虽是一次重大版本更新，但后续工作仍将继续。本次发布既让更多社区成员能够使用此前积累的改进和新增内容，也为今后的完善奠定了基础。后续工作可能包括由社区在核心要求集的基础上编写实施指南和验证指南。

ASVS 5.0 旨在为安全软件开发提供可靠基础。我们诚邀社区采用本标准、参与贡献，并在此基础上继续完善相关内容，共同推动应用安全发展。

什么是 ASVS?

应用安全验证标准（Application Security Verification Standard, ASVS）为 Web 应用和服务定义了安全要求。对于希望设计、开发、维护安全应用，或评估应用安全性的人来说，它都是一份有价值的资源。

本章概述使用 ASVS 时需要理解的关键内容，包括它的范围、基于优先级的分级结构，以及该标准的主要使用场景。

ASVS 的范围

ASVS的范围可以从名称中的四个词来理解：应用、安全、验证和标准。这四个方面共同决定了哪些要求应纳入本标准。ASVS 的总体目标是明确应用必须落实的安全原则。文档要求也在其范围之内，因为相关文档是正确实现安全要求的基础。

对攻击者来说，并不存在所谓的范围边界。因此，在评估 ASVS 要求时，也应结合应用生命周期其他方面的指南，包括 CI/CD 流程、托管环境和运维活动。

应用

ASVS将“应用”定义为正在开发的软件产品，安全控制必须集成到其中。ASVS 不规定开发生命周期活动，也不规定如何通过 CI/CD 流水线构建应用；它规定的是产品本身必须达到的安全结果。

Web 应用防火墙（WAF）、负载均衡器和代理等组件会转发、修改或检查 HTTP 流量。由于某些安全控制需要依靠这些组件实现，因此在相关场景下，可将它们视为应用的一部分。例如，验证响应缓存、速率限制，以及按来源或目的地址限制出入站连接等要求时，都应将这些组件纳入评估范围。

相反，ASVS 通常不包含那些与应用没有直接关系，或配置不属于应用职责范围的要求。例如，DNS 问题通常由独立团队或职能负责。

同样，应用负责自身处理输入和生成输出的方式，而与应用或其数据交互的外部流程不属于 ASVS 的范围。例如，应用及其数据的备份通常由外部流程负责，不受应用或其开发人员直接控制。

安全

每条要求都必须对安全有明确且可验证的作用。不满足该要求会削弱应用的安全性；落实该要求，则必须能够降低某项安全风险发生的可能性或减轻其影响。

其他考虑因素，例如功能特性、代码风格或策略要求，都不属于本标准范围。

验证

要求必须可以验证，而且验证结果必须能得出“失败”或“通过”的结论。

标准

ASVS是一套可用于判断应用是否符合标准的安全要求。因此，要求只规定必须达到的安全目标。实现指南、测试方法等相关内容，可以在 ASVS 的基础上另行编写，或通过映射与 ASVS 关联。

具体来说，OWASP 有许多项目，ASVS 有意避免与其他项目的内容重叠。例如，开发人员可能会问：“在我的技术栈或环境中，如何实现某条具体要求？”这类问题应由 Cheat Sheet Series 项目覆盖。验证方可能会问：“在这个环境中，如何测试这条要求？”这类问题应由 Web Security Testing Guide 项目覆盖。

ASVS 并非只面向安全专家，但读者仍需具备理解相关内容所需的技术知识，或能够自行查阅和理解相关概念。

要求

ASVS 特意使用“要求”一词，因为其中规定的内容是符合本标准所必须达到的。ASVS 正文只收录必须满足的要求，不把“应该做到”的建议作为合规条件。

如果某项内容只是众多可选方案之一，或仅涉及代码风格，就不属于这里所说的“要求”。

每条 ASVS 要求都针对具体的安全原则，并尽量避免限定某种技术或实现方式；同时，要求本身也应体现设置该要求的原因。因此，ASVS 不会围绕某一种验证方法或实现方案来编写要求。

已记录的安全决策

在软件安全中，尽早规划安全设计和拟采用的机制，有助于在最终产品或功能中获得更一致、更可靠的实现。

有些安全要求实现起来较为复杂，而且具体做法高度依赖应用自身的需求，例如权限管理、输入验证，以及针对不同敏感级别数据采取的保护措施。

因此，ASVS 不会笼统地规定“所有数据都必须加密”，也不会试图用一条要求覆盖所有场景。对于这类控制，ASVS 要求应用开发方以文档形式记录所作的决策和具体配置。验证方可以先判断这些决策是否合理，再对照文档检查实际实现是否符合预期。

这些文档要求用于记录开发组织在实现特定安全要求时作出的决策。

文档要求一律放在各章第一节，但并非每章都有。每条文档要求都会对应一条实施要求，用于确认文档中记录的决策已经落实。需要注意的是，检查文档是否存在和检查实际实现是否到位，是两项不同的验证工作。

设置文档要求主要有两个原因。首先，许多安全要求都要落实一组具体规则，例如允许上传哪些文件类型、必须执行哪些业务控制，以及某个字段可以包含哪些字符。每个应用采用的规则各不相同，因此 ASVS 无法统一规定这些规则，速查表或更详细的说明也无法解决这一问题。如果不把这些决策记录下来，验证方也就无法判断实际实现是否正确。

其次，有些要求需要给应用开发方留出选择空间，以便根据实际情况应对安全问题。例如，早期 ASVS 版本对会话超时作出了非常具体的规定，但许多应用，尤其是面向普通用户的应用，会采用更宽松的超时设置，并辅以其他缓解措施。因此，文档要求明确允许开发方在这方面灵活处理。

这些决策不应由每位开发人员各自作出和记录，而应由组织统一制定、传达给开发人员，并在开发过程中贯彻执行。

在软件开发中，为新功能提供规格说明和设计方案是常规做法；要求开发人员统一使用公共组件和界面机制也很常见。同样的做法自然也适用于安全领域。

记录和落实安全决策的方式可以灵活选择。组织既可以把决策写入文档供开发人员查阅，也可以将其固化在所有开发人员都必须使用的公共代码库中。两种方式都可以达到预期目的。

应用安全验证等级

ASVS 定义了三个安全验证等级，每个等级的深度和复杂度逐级增加。总体目标是让组织先从第 1 级开始，处理最关键的安全问题，再根据组织和应用需要逐步提升到更高等级。文档和要求文本中可能使用 L1、L2、L3 表示这些等级。

每个 ASVS 等级都规定了达到该等级必须满足的安全要求。尚未达到的更高等级要求，可以作为后续改进建议。

为了避免重复要求，或避免某些要求在更高等级中不再适用，一些要求会适用于某个特定等级，但在更高等级中附加更严格的条件。

等级评估

各等级根据要求的优先级划分，优先级则结合安全要求的实施和测试经验确定。评估时主要权衡风险降低效果与实现成本，同时尽量降低采用门槛。

评估风险降低效果时，既要考虑要求对机密性、完整性和可用性的保护作用，也要判断它属于主要防线还是纵深防御措施。

项目组对评估标准和等级划分进行了充分讨论，最终方案适用于绝大多数场景，但不可能与所有情况完全吻合。因此，组织可以根据自身面临的具体风险，提前实施某些更高等级的要求。

各等级中的要求类型大致可以这样理解。

第 1 级

该等级包含保护应用时最少需要考虑的要求，是一个关键起点。它大约包含 ASVS 要求的 20%。该等级的目标是尽可能减少要求数量，以降低采用门槛。

这些要求通常是关键或基础的第一层防护要求，用于防止常见攻击；这些攻击不需要依赖其他漏洞或前置条件就可能被利用。

除第一层防护要求外，一些在更高等级影响较小的要求也放在这里，例如与密码相关的要求。这些要求对第 1 级更重要，因为更高等级还涉及多因素认证要求。

第 1 级并不一定能由没有内部文档或代码访问权限的外部测试人员通过渗透测试验证（例如“黑盒”测试），不过较少的要求数量应能让验证更容易。

第 2 级

大多数应用都应努力达到这一安全等级。ASVS 中约 50% 的要求属于 L2，这意味着应用若要符合 L2，需要实现 ASVS 中约 70% 的要求（所有 L1 和 L2 要求）。

这些要求通常涉及不太常见的攻击，或针对常见攻击的更复杂防护。它们可能仍然属于第一层防护，也可能要求攻击满足某些前置条件才会成功。

第 3 级

希望证明自身达到最高安全水平的应用应以此等级为目标。L3 约占全部要求的 30%；与 L1、L2 合并后，构成完整的 ASVS 要求集。

本等级中的要求通常是纵深防御机制，或其他有价值但较难实现的控制。

应达到哪个等级

基于优先级的等级旨在反映组织和应用的应用安全成熟度。ASVS 不会规定某个应用必须处于哪个等级；组织应分析自身风险，并根据应用敏感性以及应用用户的期望，决定自己认为应该达到的等级。

例如，只收集有限敏感数据的早期创业公司，可能会把第 1 级作为初始安全目标；但银行如果让客户相信其网上银行应用安全可靠，可能很难证明低于第 3 级是合理的。

如何使用 ASVS

ASVS 的结构

ASVS 总共包含约 350 条要求，分为 17 个章节，每个章节又进一步划分为多个小节。

章节和小节划分的目的，是便于根据应用相关性选择或过滤章节与小节。例如，对于机器到机器的 API，V3 章中与 Web 前端相关的要求并不适用。如果没有使用 OAuth 或 WebRTC，那么这些章节也可以忽略。

发布策略

ASVS版本号采用“主版本.次版本.补丁版本”的格式，数字变化反映版本的变更范围：主版本更新第一个数字，次版本更新第二个数字，补丁版本更新第三个数字。

- 主版本发布 - 全面重组，几乎所有内容都可能变化，包括要求编号。需要重新评估合规性（例如 4.0.3 -> 5.0.0）。
- 次版本发布 - 可能新增或删除要求，但整体编号保持不变。需要重新评估合规性，但通常会更容易（例如 5.0.0 -> 5.1.0）。
- 补丁版本发布 - 可能删除要求（例如重复或过时的要求），或降低要求严格程度；但符合上一版本的应用也会符合该补丁版本（例如 5.0.0 -> 5.0.1）。

以上说明专门针对 ASVS 中的要求。周边文本以及附录等其他内容的变更，不视为破坏性变更。

灵活使用 ASVS

前文提到的若干机制，例如文档要求和等级机制，使组织能够以更灵活、更贴合自身情况的方式使用 ASVS。

我们也强烈鼓励组织针对自身或所在领域创建定制分支，并根据应用特点和风险调整要求。不过，定制时必须保留可追溯性，确保同一要求编号（例如 4.1.1）在各个版本中表达相同的含义。

理想情况下，每个组织都应创建自己的定制版 ASVS，省略无关章节（例如未使用 GraphQL、WebSockets、SOAP 时，可省略这些内容）。组织专属的 ASVS 版本或补充材料，也适合提供组织内部的实现指南，说明在满足要求时应使用哪些库或资源。

如何引用 ASVS 要求

每条要求都有一个 `<chapter>.<section>.<requirement>` 格式的标识符，其中每个元素都是数字。例如：1.11.3。

- `<chapter>` 值对应要求所在章节；例如，所有 1.#.# 要求都来自“编码和净化”章节。
- `<section>` 值对应该章节内要求所在的小节；例如，所有 1.2.# 要求都位于“编码和净化”章节中的“注入预防”小节。
- `<requirement>` 值标识该章节和小节中的具体要求；例如，1.2.5 在本标准 5.0.0 版中是：

验证应用能够防范操作系统命令注入，且操作系统调用采用参数化操作系统查询，或采用适合命令行上下文的输出编码。

由于标识符可能在标准版本之间变化，因此其他文档、报告或工具最好使用以下格式：`v<version>-<chapter>.<section>.<requirement>`，其中 `version` 是 ASVS 版本标签。例如：`v5.0.0-1.2.5` 应理解为特指 5.0.0 版中“编码和净化”章节“注入预防”小节的第 5 条要求。（也可以概括为 `v<version>-<requirement_identifier>`。）

注意：格式中版本号前面的 `v` 应始终使用小写。

如果标识符中没有 `v<version>`，则默认指向最新版 ASVS。由于要求会随版本演进而变化，这种写法可能产生歧义，因此编写文档或开发工具时应始终包含版本号。

ASVS 要求列表会以 CSV、JSON 和其他格式提供，便于引用或在程序中使用。

ASVS 分支

组织可以通过选择三个等级之一来采用 ASVS，也可以创建领域专属分支，根据应用风险等级调整要求。只要能够保持可追溯性，确保要求 4.1.1 在所有版本中通过验证时都表示同一件事，就鼓励采用这种分支方式。

理想情况下，每个组织都应创建自己的定制版 ASVS，省略无关章节（例如未使用 GraphQL、Websockets、SOAP 时，可省略这些内容）。分支应以 ASVS 第 1 级作为基线，再根据应用风险推进到第 2 级或第 3 级。

ASVS 的使用场景

ASVS 可用于评估应用安全性，下一章会对此进行更深入说明。不过，ASVS（或其分支版本）还有其他若干潜在用途。

作为详细的安全架构指南

应用安全验证标准最常见的用途之一，是作为安全架构师的参考资源。关于如何构建安全的应用架构，尤其是现代应用架构，可用资源并不多。ASVS 可以帮助填补这些空白，让安全架构师能够为常见问题选择更好的控制措施，例如数据保护模式和输入验证策略。架构和文档要求在这方面尤其有用。

作为专门的安全编码参考

ASVS 可作为应用开发期间准备安全编码参考的基础，帮助开发人员在构建软件时始终考虑安全。虽然 ASVS 可以作为基础，但组织应准备自己的具体指南，确保其清晰、统一，并且最好基于安全工程师或安全架构师的指导来制定。在此基础上，也鼓励组织尽可能准备经批准的安全机制和库，使其既可在指南中引用，也可供开发人员使用。

作为自动化单元测试和集成测试指南

ASVS 的设计目标之一是高度可测试。有些验证是技术性的，而其他要求（例如架构和文档要求）可能需要进行文档或架构审查。围绕可通过技术手段验证的要求，构建针对具体且相关滥用场景的单元测试、集成测试和模糊测试，可以更容易地在每次构建时确认这些控制是否正常工作。例如，可以为登录控制器的测试套件编写额外测试，检查用户名参数是否存在常见默认用户名、账号枚举、暴力破解、LDAP 和 SQL 注入以及 XSS 等问题。同样，针对密码参数的测试也应包括常见密码、密码长度、空字节注入、移除参数、XSS 等情况。

用于安全开发培训

ASVS 也可用于定义安全软件的特征。许多“安全编码”课程其实只是伦理黑客课程，再附带少量编码建议。这未必能帮助开发人员写出更安全的代码。相反，安全开发课程可以使用 ASVS，并重点讲解 ASVS 中的正向机制，而不是只关注 Top 10 中那些“不该做”的负面事项。ASVS 的结构也为讲解保护应用时的不同主题提供了清晰顺序。

作为安全软件采购指导框架

ASVS是帮助安全软件采购或定制开发服务采购的优秀框架。采购方可以直接提出要求：希望采购的软件必须按 ASVS X 级开发，并要求供应方证明该软件满足 ASVS X 级。

在实践中应用 ASVS

不同威胁有不同动机。一些行业拥有独特的信息和技术资产，也有特定领域的监管合规要求。

强烈鼓励组织根据自身业务性质深入审视独特的风险特征，并基于这些风险和业务要求确定合适的 ASVS 等级。

评估和认证

OWASP 对 ASVS 认证和信任标志的立场

OWASP是一家坚持厂商中立原则的非营利组织，不会为任何厂商、验证方或软件提供认证。任何声称符合 ASVS 的保证、信任标志或认证，都不代表获得了 OWASP 官方认可。因此，组织应谨慎看待第三方提出的 ASVS 认证声明。

组织可以提供保证服务，但不得声称获得 OWASP 官方认证。

如何验证 ASVS 合规性

ASVS不会像测试指南那样详细规定合规验证方法，但仍有几个关键问题需要说明。

验证报告

传统渗透测试报告通常只列出未通过的项目。ASVS认证报告则应说明验证范围，汇总所有已检查的要求，列明未通过的要求，并给出问题处理建议。对于不适用的要求（例如无状态 API 中的会话管理要求），也必须在报告中明确标注。

验证范围

开发应用的组织通常不会实施全部要求，因为根据应用的功能，某些要求可能不相关，或重要性较低。验证方应明确说明验证范围，包括组织希望达到的等级以及本次实际纳入的要求。报告应以纳入验证的内容为主，而不能只罗列排除项。对于未实现且被排除的要求，验证方还应评价排除理由是否合理。

只有这样，报告使用者才能了解验证背景，并据此判断该应用在多大程度上值得信任。

认证组织可以自行选择测试方法，但应在报告中披露这些方法，并且理想情况下应可重复执行。根据应用和要求的不同，可以采用人工渗透测试、源代码分析等不同方法来验证输入验证等方面。

验证机制

验证具体的 ASVS 要求往往需要综合运用多种方法。除了使用有效凭据开展渗透测试，以覆盖应用的全部功能外，验证方可能还需要查阅文档、源代码和配置，并与开发流程的参与者沟通。验证 L2 和 L3 要求时尤其如此。通常的做法是提供有力证据和详细记录来支持验证发现，这些材料可包括工作底稿、截图、脚本和测试日志。仅运行自动化工具而不开展深入测试，不足以支持认证结论，因为每条要求都必须得到切实验证。

使用自动化来验证 ASVS 要求一直是大家关注的话题。因此，有必要澄清一些与自动化测试和黑盒测试相关的要点。

自动化安全测试工具的作用

将动态和静态应用安全测试工具（DAST 和 SAST）正确集成到构建流水线后，可以发现一些本不应出现的安全问题。但如果没有经过妥善配置和调优，这些工具往往无法达到所需的覆盖范围；过多的误报和无关结果还会妨碍真正安全问题的发现和处理。

这类工具或许能够覆盖一些较基础、较直接的技术要求，例如与输出编码或净化有关的要求。但必须注意，这些工具无法完整验证许多较复杂的 ASVS 要求，或与业务逻辑和访问控制有关的要求。

较复杂的要求也可能实现自动验证，但需要针对具体应用编写测试。这些测试与组织现有的单元测试和集成测试类似，因此可以利用现有自动化测试基础设施来编写 ASVS 专项测试。这样做虽然需要前期投入，却能持续检查相关要求，长期收益十分明显。

总之，可自动化测试并不等同于运行现成工具。

渗透测试的作用

4.0 版曾针对“黑盒”测试（无法查看文档和源代码的测试）优化 L1，但即使在当时，标准也明确指出这种方式无法提供有效保证，应积极劝阻采用这种方式。

在无法获取必要补充信息的情况下开展测试，是一种低效且无效的安全验证方式。他们既无法审查源代码，也难以识别潜在威胁和缺失的控制措施，还会失去在较短时间内开展深入测试的机会。

强烈建议开展以文档或源代码为基础的（混合）渗透测试，使测试人员能够与应用开发人员充分沟通，并完整查阅应用文档，而不是采用传统渗透测试方式。要验证许多 ASVS 要求，这是必要的。

与 v4.x 相比的变化

引言

熟悉本标准 4.x 版本的用户，可能会发现回顾 5.0 版引入的关键变化很有帮助。这些变化包括内容、范围和底层理念的更新。

在 4.0.3 版的 286 条要求中，只有 11 条保持不变，另有 15 条仅做了不改变含义的轻微语法调整。总计有 109 条要求（38%）在 5.0 版中不再作为独立要求存在，其中 50 条直接删除，28 条因重复被移除，31 条被合并到其他要求。其余要求都以某种方式进行了修订。即便内容没有实质性变化的要求，也会因为重新排序或结构调整而拥有不同标识符。

为了便于用户迁移到 5.0 版，标准提供了映射文档，用于追踪 4.x 与 5.0 要求之间的对应关系。映射文档独立于正式版本发布，必要时可以单独更新或补充说明。

要求理念

范围和重点

4.x 版本包含了一些不符合标准预期范围的要求，这些要求已经被移除。不符合 5.0 范围标准或不可验证的要求也已被排除。

更强调安全目标，而不是具体机制

在 4.x 版本中，许多要求关注具体机制，而不是底层安全目标。在 5.0 版中，要求围绕安全目标展开，只有当某种机制是唯一可行方案时才会引用具体机制，或把具体机制作为示例或补充说明。

同一安全目标往往可以通过多种方式实现。采用这种写法，可以避免因不必要地限定实现方式而影响组织的灵活选择。

此外，针对同一安全关注点的要求，也已在适当情况下进行了合并。

已记录的安全决策

“已记录的安全决策”看似是 5.0 版新增的概念，其实源自 4.0 版中有关策略执行和威胁建模的要求。早期版本的一些要求已经隐含要求组织先开展分析，再据此实施安全控制，例如确定允许建立哪些网络连接。

5.0 版将这些内容明确写成文档要求，确保实施和验证时能够取得所需信息，也使要求更加清晰、可执行、可验证。

结构变化和新增章节

5.0 版中有几个章节引入了全新内容：

- OAuth 和 OIDC - 鉴于这些协议在访问委派和单点登录中的广泛采用，标准新增了专门要求，以覆盖开发人员可能遇到的多种场景。该领域未来可能会发展成独立标准，类似早期版本中对移动和 IoT 要求的处理。
- WebRTC - 随着这一技术越来越流行，其独特的安全考虑和挑战现在由专门章节覆盖。

标准还重新整理了章节和小节，使相关要求能够集中归类。

这种重组带来了若干新增章节：

- 自包含令牌 - 自包含令牌过去归入会话管理，现在被视为一种独立机制，也是无状态通信（例如 OAuth 和 OIDC）中的基础元素。由于它们具有独特安全影响，标准用专门章节处理，并在 5.x 版中引入了一些新要求。
- Web 前端安全 - 随着基于浏览器的应用越来越复杂，以及仅 API 架构兴起，前端安全要求被拆分为独立章节。
- 安全编码和架构 - 一些无法归入现有章节的通用安全实践新要求，被集中放在这里。

5.0 版中的其他组织结构调整，主要是为了澄清意图。例如，输入验证要求被移到业务逻辑旁边，体现其在执行业务规则中的作用，而不是继续与净化和编码放在一起。

原 V1 架构章节已被移除。它最初的小节包含超出范围的要求，后续小节则已重新分配到相关章节，并在必要时去重和澄清。

移除与其他标准的直接映射

标准正文不再直接列出与其他标准的映射。项目计划改为将 ASVS 映射到 OWASP Common Requirement Enumeration (CRE)，再由 CRE 建立 ASVS 与其他 OWASP 项目及外部标准之间的联系。

与 CWE 和 NIST 的直接映射不再维护，原因如下。

降低与 NIST 数字身份指南的耦合

NIST 数字身份指南 (SP 800-63) 长期以来一直是认证和授权控制的参考。4.x 版本中，某些章节与 NIST 的结构和术语高度一致。

虽然这些指南仍是重要参考，但严格对齐也带来了挑战，包括使用的术语不够广为人知、类似要求重复，以及映射不完整。5.0 版不再采用这种方式，以提升清晰度和相关性。

不再直接映射通用弱点枚举 (Common Weakness Enumeration, CWE)

通用弱点枚举 (Common Weakness Enumeration, CWE) 为软件安全弱点提供了一套实用的分类体系。不过，有些 CWE 条目仅表示类别，一条要求也难以准确映射到单个 CWE；此外，4.x 版本还存在部分映射不准确的问题。因此，5.0 版不再维护与 CWE 的直接映射。

重新思考等级定义

4.x 版本将等级描述为 L1 (“最低”，Minimum)、L2 (“标准”，Standard) 和 L3 (“高级”，Advanced)，并暗示所有处理敏感数据的应用至少应满足 L2。

5.0 版解决了这种做法中的若干问题，下文会逐一说明。

从实际角度看，4.x 版本使用勾选标记作为等级指示，而 5.x 在标准的所有格式中都使用简单数字，包括 Markdown、PDF、DOCX、CSV、JSON 和 XML。为保持向后兼容，仍会生成使用勾选标记的旧版 CSV、JSON 和 XML 输出。

更容易入门的等级

反馈显示，第 1 级要求数量过多（约 120 条），再加上它被称为对多数应用来说“不够好”的“最低”等级，阻碍了采用。5.0 版希望降低这一门槛，将第 1 级主要定义为第一层防护要求，使该等级的要求更清晰、数量更少。用数字说明：v4.0.3 中共有 278 条要求，其中 128 条为 L1，占 46%；5.0.0 中共有 345 条要求，其中 70 条为 L1，占 20%。

可测试性谬误

在 4.x 版本中，选择第 1 级控制时，一个关键因素是它们是否适合通过“黑盒”外部渗透测试评估。然而，这种方式并未完全符合第 1 级作为最低安全控制集合的意图。一些用户认为第 1 级不足以保护应用，另一些用户则发现它太难测试。

以可测试性作为等级划分标准具有相对性，有时也会产生误导。某条要求可以验证，并不意味着能够轻松或自动完成验证。此外，最容易测试的要求，未必最能提高安全性，也未必最容易实现。

因此，在 5.0 版中，等级决策主要基于风险降低，同时也考虑实现成本。

不只是风险

基于风险并规定某些应用必须达到特定等级的做法，已经被证明过于僵化。实践中，安全控制的优先级和实现取决于多个因素，包括风险降低效果和实现所需成本。

因此，鼓励组织根据自身成熟度以及希望向用户传达的信息，达到其认为自身应该达到的等级。

V1 编码和净化

控制目标

本章讨论与不安全处理不可信数据有关的常见 Web 应用安全弱点。这类弱点可能导致多种技术漏洞，其中不可信数据会按照相应解释器的语法规则被解析。

对于现代 Web 应用，最好始终使用更安全的 API，例如参数化查询、自动转义机制或模板框架。否则，谨慎执行输出编码、转义或净化对应用安全至关重要。

输入验证是一种纵深防御机制，用来防止意外或危险内容。不过，由于它的主要目的是确保传入内容符合功能和业务预期，因此相关要求位于“验证和业务逻辑”章节。

V1.1 编码和净化架构

以下各节针对不同语法和解释器，说明如何安全处理不可信内容。本节则规定这些处理应在何时、何处执行。数据在存储时应保持原始形式，不应存储为已经编码或转义的形式（例如经过 HTML 编码的文本），否则容易发生重复编码。

#	描述	级别
1.1.1	验证输入最多只经过一次解码或反转义，并转换为规范形式。只有在预期输入本身采用了相应编码时，才可以进行解码。解码或反转义必须先于其他输入处理；例如，不得在输入验证或净化之后再进行解码或反转义。	2

#	描述	级别
1.1.2	验证应用将输出编码和转义作为数据交由目标解释器使用之前的最后一个处理步骤，或者由目标解释器自身执行这些操作。	2

V1.2 注入预防

在接近或紧邻潜在危险上下文的位置进行输出编码或转义，对任何应用的安全都至关重要。通常，输出编码和转义的结果不会被持久化保存，而是用于使输出能够立即在相应解释器中安全使用。过早执行这些操作可能导致内容格式错误，或使编码或转义失效。

许多软件库都提供能够自动完成这些处理的安全函数。即便如此，仍须确认所用函数适合数据所在的具体上下文。

#	描述	级别
1.2.1	验证应用在生成 HTTP 响应、HTML 文档或 XML 文档时，根据数据所在的具体上下文进行编码。例如，针对 HTML 元素、HTML 属性、HTML 注释、CSS 或 HTTP 头字段分别编码相应的特殊字符，防止数据改变消息或文档结构。	1
1.2.2	验证动态构建 URL 时，根据数据所在位置对不可信数据进行编码。例如，对查询参数或路径参数采用 URL 编码或 base64url 编码。同时，只允许使用安全的 URL 协议，不得允许 javascript:、data: 等危险协议。	1
1.2.3	验证应用动态生成 JavaScript 内容（包括 JSON）时，会根据 JavaScript 或 JSON 上下文，对插入的数据进行适当编码或转义，防止不可信数据改变消息或文档结构，从而避免 JavaScript 注入和 JSON 注入。	1
1.2.4	验证数据检索或数据库查询（例如 SQL、HQL、NoSQL、Cypher）采用参数化查询、ORM、实体框架或其他有效措施，以防范 SQL 注入及其他数据库注入攻击。编写存储过程时也必须满足此要求。	1
1.2.5	验证应用能够防范操作系统命令注入，且操作系统调用采用参数化操作系统查询，或采用适合命令行上下文的输出编码。	1
1.2.6	验证应用能够防范 LDAP 注入漏洞，或已实施用于防止 LDAP 注入的特定安全控制。	2
1.2.7	验证应用使用参数化查询或预编译查询来防范 XPath 注入。	2
1.2.8	验证 LaTeX 处理器采用了安全配置，例如不启用 <code>--shell-escape</code> ；同时使用命令允许列表来防范 LaTeX 注入。	2
1.2.9	验证应用对正则表达式中的特殊字符进行转义（通常使用反斜杠），以防止这些字符被误解释为元字符。	2

#	描述	级别
1.2.10	验证应用能够防范 CSV 和公式注入。导出 CSV 内容时，应用必须遵循 RFC 4180 第 2.6 和 2.7 节定义的转义规则。此外，导出为 CSV 或其他电子表格格式（例如 XLS、XLSX 或 ODF）时，如果特殊字符（包括 ‘=’、‘+’、‘-’、‘@’、‘\t’（制表符）和 ‘\0’（空字符））出现在字段值的第一个字符位置，必须用单引号进行转义。	3

注意：参数化查询或 SQL 转义并不能解决所有问题。表名、列名（包括 ORDER BY 子句中的列名）等查询结构无法通过转义来保证安全。如果把用户提供的值用于这些位置，即使经过转义，也可能造成查询失败或 SQL 注入。

V1.3 净化

需要在危险上下文中使用不可信内容时，首选方法是根据上下文进行编码或转义。这样既能保留原内容的含义，又能保证它在该上下文中安全使用。上一节对此作了详细说明。

如果无法做到这一点，就需要进行净化，移除潜在危险字符或内容。在某些情况下，这可能会改变输入的语义，但出于安全原因，可能没有其他选择。

#	描述	级别
1.3.1	验证所有来自 WYSIWYG 编辑器或类似来源的不可信 HTML，都经过知名且安全的 HTML 净化库或框架功能处理。	1
1.3.2	验证应用避免使用 eval()、Spring Expression Language (SpEL) 等动态代码执行功能。如果确实没有替代方案，则必须先净化其中包含的所有用户输入，再执行动态代码。	1
1.3.3	验证数据进入潜在危险的上下文之前已经过净化，并且符合该上下文的安全限制。例如，只允许安全字符，并截断过长的输入。	2
1.3.4	验证用户提供的可缩放矢量图形 (SVG) 内容经过验证或净化，只保留对应用安全的标签和属性（例如绘图相关内容），不得包含脚本、foreignObject 等危险内容。	2
1.3.5	验证应用对用户提供的可脚本化内容或表达式模板内容进行净化，或者彻底禁用这类内容，例如 Markdown、CSS、XSL 样式表、BBCode 等。	2
1.3.6	验证应用能够防范服务端请求伪造 (SSRF) 攻击：在使用不可信数据调用其他服务前，先按协议、域名、路径和端口允许列表验证这些数据，并净化潜在危险字符。	2
1.3.7	验证应用禁止使用不可信输入来构建模板，以防范模板注入。如果没有其他可行方案，则必须对模板创建过程中动态插入的所有不可信输入进行净化或严格验证。	2
1.3.8	验证不可信输入在用于 Java Naming and Directory Interface (JNDI) 查询之前已经过适当净化，并且 JNDI 本身采用了安全配置，以防范 JNDI 注入。	2
1.3.9	验证内容发送到 memcache 之前已经过净化，以防范注入攻击。	2
1.3.10	验证在使用时可能以非预期或恶意方式解析的格式字符串，在处理之前已经过净化。	2

#	描述	级别
1.3.11	验证用户输入在传递给邮件系统之前已经过净化，以防范 SMTP 或 IMAP 注入。	2
1.3.12	验证正则表达式中没有什么可能引发指数级回溯的结构，并对不可信输入进行净化，以降低 ReDoS 或失控正则表达式攻击的风险。	3

V1.4 内存、字符串和非托管代码

以下要求处理不安全内存使用带来的风险，这通常适用于使用系统语言或非托管代码的应用。

部分要求可以通过编译器选项来落实。例如，启用缓冲区溢出保护、栈随机化、数据执行保护和相关警告；一旦发现不安全的指针、内存、格式字符串、整数或字符串操作，就让构建失败。

#	描述	级别
1.4.1	验证应用采用内存安全的字符串操作、更安全的内存复制方法和指针运算，以检测或防止栈溢出、缓冲区溢出和堆溢出。	2
1.4.2	验证应用通过符号检查、范围检查和输入验证来防止整数溢出。	2
1.4.3	验证动态分配的内存和资源在使用完毕后得到释放；同时删除或置空指向已释放内存的引用和指针，以防止悬空指针和释放后使用漏洞。	2

V1.5 安全反序列化

反序列化是把存储或传输形式的数据转换为应用对象的过程。这个过程曾引发多种代码注入漏洞，因此必须谨慎、安全地处理。

尤其要注意，有些反序列化方法已被编程语言或框架文档认定为不安全，无法使其在处理不可信数据时安全运行。应对每一种正在使用的反序列化机制进行审慎评估。

#	描述	级别
1.5.1	验证 XML 解析器采用严格的安全配置，并禁用外部实体解析等危险功能，以防范 XML 外部实体（XXE）攻击。	1
1.5.2	验证应用在反序列化不可信数据时，强制采用安全的输入处理方式，例如设置对象类型允许列表，或限制客户端能够指定的对象类型，以防范反序列化攻击。不得使用已经明确认定为不安全的反序列化机制来处理不可信输入。	2

#	描述	级别
1.5.3	验证应用使用多个解析器处理同一类数据时（例如使用不同的 JSON、XML 或 URL 解析器），各解析器采用一致的解析规则和字符编码。这样可以避免 JSON 互操作性漏洞，也可以防止攻击者利用不同解析器对 URI 或文件的解释差异实施远程文件包含（RFI）或服务端请求伪造（SSRF）攻击。	3

参考资料

更多信息，另见：

- OWASP LDAP Injection Prevention Cheat Sheet
- OWASP Cross Site Scripting Prevention Cheat Sheet
- OWASP DOM Based Cross Site Scripting Prevention Cheat Sheet
- OWASP XML External Entity Prevention Cheat Sheet
- OWASP Web Security Testing Guide: Client-Side Testing
- OWASP Java Encoding Project
- DOMPurify - Client-side HTML Sanitization Library
- RFC4180 - Common Format and MIME Type for Comma-Separated Values (CSV) Files

关于反序列化或解析问题的更多信息，请参见：

- OWASP Deserialization Cheat Sheet
- An Exploration of JSON Interoperability Vulnerabilities
- Orange Tsai - A New Era of SSRF Exploiting URL Parser In Trending Programming Languages

V2 验证和业务逻辑

控制目标

本章要求接受验证的应用达到以下总体目标：

- 应用接收的输入符合业务或功能预期。
- 业务逻辑流程按预期顺序逐步处理，不能被绕过。
- 业务逻辑包含限制和控制，用于检测并防止自动化攻击，例如连续进行小额转账，或一次添加一个好友直到添加一百万个。
- 高价值业务逻辑流程已考虑滥用场景和恶意行为者，并具备防范欺骗、篡改、信息泄露和权限提升攻击的保护措施。

V2.1 验证和业务逻辑文档

验证规则和业务逻辑文档应清楚说明业务限制、输入验证规则，以及多个相关数据项之间应满足的逻辑关系，让开发和测试人员明确应用必须实现哪些约束。

#	描述	级别
2.1.1	验证应用文档明确规定了输入验证规则，包括如何按照预期结构检查数据是否有效。规则既可以针对信用卡号、电子邮件地址、电话号码等常见格式，也可以针对应用内部的数据格式。	1
2.1.2	验证应用文档明确规定了如何检查多个相关数据项在逻辑和上下文上是否一致，例如市郊地区与邮政编码是否匹配。	2
2.1.3	验证应用文档明确规定了业务逻辑限制和验证要求，包括针对单个用户的限制，以及适用于整个应用的全局限制。	2

V2.2 输入验证

有效的输入验证控制应根据应用预期接收的数据类型，落实相应的业务或功能要求。这可以保证良好的数据质量，并缩小攻击面。但在其他组件中使用这些数据或将其用于输出时，仍需进行正确的编码、参数化或净化，输入验证不能消除或取代这些需求。

在此语境下，“输入”可能来自多种来源，包括 HTML 表单字段、REST 请求、URL 参数、HTTP 头字段、Cookie、磁盘文件、数据库和外部 API。

例如，业务规则可能要求某个输入必须是小于 100 的数字。功能规则也可能为控制循环次数的参数设定上限，防止数值过大造成过量计算，进而引发拒绝服务。

本标准没有明确强制要求采用模式验证。不过，对于使用 JSON 或 XML 的 HTTP API 或其他接口，模式验证可能是实现完整验证覆盖的最有效机制。

请注意以下关于模式验证的要点：

- JSON Schema 验证规范的“已发布版本”被认为可用于生产环境，但严格来说并不算“稳定”。使用 JSON Schema 验证时，应确保与下列要求中的指导不存在缺口。
- 正在使用的任何 JSON Schema 验证库，也应在标准正式确定后持续监控，并在必要时更新。
- 不应使用 DTD 验证，并且应禁用框架的 DTD 求值，以避免 DTD 相关 XXE 攻击问题。

#	描述	级别
2.2.1	验证对输入进行验证，以落实该输入对应的业务或功能要求。应根据值、模式和范围的允许列表进行正向验证，或按照预先定义的规则，将输入与预期结构及逻辑限制进行比较。L1 可以重点验证用于作出特定业务或安全决策的输入；L2 及以上应对所有输入进行验证。	1

#	描述	级别
2.2.2	验证应用在可信服务层强制执行输入验证。客户端验证有助于改善使用体验，应予鼓励，但不得将其作为安全控制加以依赖。	1
2.2.3	验证应用按照预先定义的规则，检查多个相关数据项组合在一起后是否合理。	2

V2.3 业务逻辑安全

本节说明如何确保应用按照预期执行业务流程，并防范攻击者利用流程设计或业务逻辑缺陷。

#	描述	级别
2.3.1	验证应用按照预期的连续步骤顺序为同一用户处理业务逻辑流程，不允许跳过任何步骤。	1
2.3.2	验证应用按照文档规定落实了各项业务逻辑限制，防止攻击者利用业务逻辑缺陷。	2
2.3.3	验证业务逻辑层使用事务来执行相关操作，使整组操作要么全部成功，要么回滚到操作前的正确状态。	2
2.3.4	验证业务逻辑层采用了适当的锁定机制，防止攻击者通过操纵应用流程重复预订数量有限的资源，例如剧院座位或配送时段。	2
2.3.5	验证高价值业务流程必须经过多人审批，以防止未经授权或误操作。此类流程可能包括但不限于大额资金转账、合同审批、访问涉密信息，或在制造过程中绕过安全保护。	3

V2.4 反自动化

本节包含反自动化控制，要求交互方式符合人工操作特征，并防止过量的自动化请求。

#	描述	级别
2.4.1	验证应用采用了反自动化控制，防止应用功能被过量调用。这类滥用可能造成数据外泄、产生大量垃圾数据、耗尽配额、规避速率限制、拒绝服务，或大量消耗高成本资源。	2
2.4.2	验证业务流程对操作节奏作出限制，使其符合真实用户的操作速度，防止业务操作被异常快速地连续提交。	3

参考资料

更多信息，另见：

- OWASP Web Security Testing Guide: Input Validation Testing
- OWASP Web Security Testing Guide: Business Logic Testing

- 反自动化可以通过多种方式实现，包括使用 OWASP Automated Threats to Web Applications
- OWASP Input Validation Cheat Sheet
- JSON Schema

V3 Web 前端安全

控制目标

本章重点防范针对 Web 前端的攻击，不适用于纯机器间通信的系统。

V3.1 Web 前端安全文档

本节概述应用文档中应明确规定的浏览器安全功能。

#	描述	级别
3.1.1	验证应用文档列明了浏览器必须支持的安全功能，例如 HTTPS、HTTP 严格传输安全 (HSTS)、内容安全策略 (CSP) 及其他 HTTP 安全机制。文档还必须说明浏览器不支持其中某项功能时，应用应如何处理，例如警告用户或拒绝访问。	3

V3.2 非预期内容解释

如果浏览器在错误的上下文中渲染内容或调用功能，就可能显示甚至执行恶意内容。

#	描述	级别
3.2.1	验证应用采用了安全控制，防止浏览器在错误的上下文中渲染 HTTP 响应中的内容或功能，例如直接访问 API、用户上传的文件或其他资源时。可采用的措施包括：只有当 Sec-Fetch-* 等 HTTP 请求头表明上下文正确时才返回内容；在 Content-Security-Policy 头中使用 sandbox 指令；或者在 Content-Disposition 头中指定 attachment。	1
3.2.2	验证只需显示为文本而不应渲染成 HTML 的内容，会通过安全渲染函数（例如 createTextNode 或 textContent）处理，防止其中的 HTML 或 JavaScript 被意外执行。	1
3.2.3	验证客户端 JavaScript 通过显式声明变量、进行严格类型检查、不在 document 对象上存储全局变量，以及隔离命名空间等方式，防范 DOM Clobbering。	3

V3.3 Cookie 设置

本节说明如何安全配置敏感 Cookie，以尽可能确认它们确实由应用创建，并防止其内容泄露或遭到篡改。

#	描述	级别
3.3.1	验证 Cookie 已设置 Secure 属性；如果 Cookie 名称未使用 __Host- 前缀，则必须使用 __Secure- 前缀。	1
3.3.2	验证每个 Cookie 都根据自身用途设置了合适的 SameSite 属性值，以降低用户界面伪装攻击和浏览器请求伪造攻击（通常称为跨站请求伪造，即 CSRF）的风险。	2
3.3.3	验证 Cookie 名称使用 __Host- 前缀，除非这些 Cookie 被明确设计为与其他主机共享。	2
3.3.4	验证当 Cookie 的值不应由客户端脚本访问时（例如会话令牌），该 Cookie 设置了 HttpOnly 属性，并且该值只能通过 Set-Cookie 头字段传输给客户端。	2
3.3.5	验证应用写入的 Cookie，其名称和值合计不超过 4096 字节。浏览器不会保存过大的 Cookie，也不会在后续请求中发送它，这可能导致依赖该 Cookie 的功能无法使用。	3

V3.4 浏览器安全机制头

本节说明 HTTP 响应应设置哪些安全头，以便浏览器在处理响应时启用相应的安全功能和限制。

#	描述	级别
3.4.1	验证所有响应都包含 Strict-Transport-Security 头字段，以强制执行 HTTP 严格传输安全（HSTS）策略。必须定义至少 1 年的最大有效期；对于 L2 及以上，策略还必须适用于所有子域名。	1
3.4.2	验证应用将跨源资源共享（CORS）的 Access-Control-Allow-Origin 响应头字段设置为固定值，或使用 Origin 请求头字段的值时，对照可信源允许列表验证该值。需要使用 Access-Control-Allow-Origin: * 时，响应中不得包含任何敏感信息。	1
3.4.3	验证 HTTP 响应包含 Content-Security-Policy 头，并通过其中的指令限制浏览器只能加载和执行可信内容或资源，从而限制恶意 JavaScript 的执行。最低要求是制定全局策略：包含 object-src 'none' 和 base-uri 'none'，并为其他资源定义允许列表，或者采用 nonce 或哈希。L3 应用必须针对每个响应定义使用 nonce 或哈希的策略。	2
3.4.4	验证所有 HTTP 响应都包含 X-Content-Type-Options: nosniff 头字段，禁止浏览器通过内容嗅探来猜测 MIME 类型。响应的 Content-Type 必须与目标资源类型相符；例如，样式资源只有在响应类型为 text/css 时才会被浏览器接受。设置该头字段还会启用浏览器的跨源读取阻止（CORB）功能。	2
3.4.5	验证应用设置了来源信息策略（referrer policy），防止技术敏感数据通过 Referrer 请求头泄露给第三方服务。可以通过 Referrer-Policy 响应头或 HTML 元素属性来设置。敏感数据可能包括 URL 路径和查询参数；对于非公开的内部应用，还可能包括主机名。	2

#	描述	级别
3.4.6	验证 Web 应用在每个 HTTP 响应的 Content-Security-Policy 头中设置了 frame-ancestors 指令。默认情况下应禁止嵌入应用内容，仅在必要时允许嵌入特定资源。虽然浏览器仍支持 X-Frame-Options，但该头已经过时，不应再依赖它。	2
3.4.7	验证 Content-Security-Policy 头指定了策略违规报告的接收位置。	3
3.4.8	验证所有会触发文档渲染的 HTTP 响应（例如 Content-Type 为 text/html 的响应）都包含 Cross-Origin-Opener-Policy 头字段，并根据需要使用 same-origin 或 same-origin-allow-popups 指令。这样可以防止攻击者滥用不同页面对 Window 对象的共享访问，例如实施标签页劫持（tabnabbing）或框架计数（frame counting）攻击。	3

V3.5 浏览器源隔离

服务端收到敏感功能的请求时，应用必须确认请求来自应用自身或可信方，而不是攻击者伪造的请求。

此处的敏感功能可以包括接受已认证和未认证用户的表单提交（例如认证请求）、改变状态的操作，或消耗大量资源的功能（例如数据导出）。

主要防护机制包括浏览器的同源策略（针对 JavaScript）和 Cookie 的 SameSite 规则。CORS 预检也是常见的防护手段：对设计为允许跨源调用的端点，这一机制至关重要；对不应跨源调用的端点，它也能帮助阻止伪造请求。

#	描述	级别
3.5.1	验证应用在不依赖 CORS 预检来保护敏感功能时，会检查请求是否确实来自应用自身。可采用并验证防伪令牌，或者要求请求携带不属于 CORS 安全列表的额外 HTTP 头。这样可以防范浏览器请求伪造攻击，即通常所说的跨站请求伪造（CSRF）。	1
3.5.2	验证应用依赖 CORS 预检来保护敏感功能时，任何不会触发预检的请求都无法调用该功能。为此，应用可能需要检查 Origin 和 Content-Type 请求头，或要求请求携带不属于 CORS 安全列表的额外头字段。	1
3.5.3	验证访问敏感功能的 HTTP 请求使用合适的 HTTP 方法，例如 POST、PUT、PATCH 或 DELETE，而不是 HTTP 规范定义为“安全”的方法，例如 HEAD、OPTIONS 或 GET。或者，可以严格验证 Sec-Fetch-* 请求头字段，确保请求并非来自不合适的跨源调用、导航请求，或资源加载（例如图片源）等非预期来源。	1
3.5.4	验证不同应用分别托管在不同的主机名下，以充分利用同源策略和 Cookie 的主机名限制，约束一个源中的文档或脚本与另一个源中的资源交互。	2
3.5.5	验证通过 postMessage 接口收到消息时，会检查消息来源和格式；来源不可信或格式不合法的消息必须丢弃。	2
3.5.6	验证应用任何位置都未启用 JSONP 功能，以避免跨站脚本包含（XSSI）攻击。	3

#	描述	级别
3.5.7	验证需要授权的数据不会包含在脚本资源响应（例如 JavaScript 文件）中，以防止跨站脚本包含（XSSI）攻击。	3
3.5.8	验证只有在符合预期时，才允许以用户身份加载或嵌入需要认证的资源，例如图片、视频、脚本和其他文档。可以严格检查 Sec-Fetch-* 请求头，排除非预期的跨源调用；也可以设置严格的 Cross-Origin-Resource-Policy 响应头，让浏览器阻止不符合策略的内容。	3

V3.6 外部资源完整性

本节为在第三方站点上安全托管内容提供指导。

#	描述	级别
3.6.1	验证 JavaScript 库、CSS、Web 字体等客户端资源只有同时满足以下条件时，才托管在外部站点（例如 CDN）：资源是静态的、带有明确版本，并通过子资源完整性（SRI）校验其完整性。无法满足这些条件时，应针对每项资源记录安全决策，说明理由。	3

V3.7 其他浏览器安全考虑

本节包含客户端浏览器安全所需的各类安全控制和现代浏览器安全功能。

#	描述	级别
3.7.1	验证应用只使用仍受支持且被认为安全的客户端技术。不符合此要求的技术示例包括 NSAPI 插件、Flash、Shockwave、ActiveX、Silverlight、NACL 或客户端 Java 小程序。	2
3.7.2	验证应用只有在目标主机名或域名位于允许列表中时，才自动把用户重定向到应用控制范围之外。	2
3.7.3	验证用户将被重定向到应用控制范围之外的 URL 时，应用会事先提示用户，并允许用户取消跳转。	3
3.7.4	验证应用的顶级域名（例如 site.tld）已经加入 HSTS 公共预加载列表。这样，主流浏览器会内置该应用必须使用 TLS 的规则，而不只是依赖 Strict-Transport-Security 响应头。	3
3.7.5	验证浏览器不支持应用所需的安全功能时，应用按照文档规定进行处理，例如警告用户或拒绝访问。	3

参考资料

更多信息，另见：

- Set-Cookie __Host- prefix details
- OWASP Content Security Policy Cheat Sheet
- OWASP Secure Headers Project
- OWASP Cross-Site Request Forgery Prevention Cheat Sheet
- HSTS Browser Preload List submission form
- OWASP DOM Clobbering Prevention Cheat Sheet

V4 API 和 Web 服务

控制目标

对于向 Web 浏览器或其他调用方开放 API 的应用（通常使用 JSON、XML 或 GraphQL），需要考虑若干特有的安全问题。本章介绍应采用的相关安全配置和机制。

认证、会话管理和输入验证等其他章节的要求同样适用于 API。因此，理解和测试本章时，必须结合标准中的其他相关要求。

V4.1 通用 Web 服务安全

本节介绍 Web 服务普遍适用的安全要求，包括最基本的安全配置和防护措施。

#	描述	级别
4.1.1	验证每个包含消息正文的 HTTP 响应都带有 Content-Type 头，且其值与实际响应内容一致。对于 IANA Media Types 规定应包含字符集的类型（例如 text/、/+xml 和 /xml），还必须通过 charset 参数指定安全的字符编码，例如 UTF-8 或 ISO-8859-1。	1
4.1.2	验证只有供用户通过浏览器访问的端点，才会自动把 HTTP 请求重定向到 HTTPS。其他服务和端点不得进行这种透明重定向。否则，客户端误用未加密 HTTP 发送敏感数据后，自动重定向可能掩盖已经发生的数据泄露。	2
4.1.3	验证应用所使用且由中间层设置的任何 HTTP 头字段，例如负载均衡器、Web 代理或 BFF 服务设置的头字段，均不能被最终用户覆盖。此类头字段可能包括 X-Real-IP、X-Forwarded-* 或 X-User-ID。	2
4.1.4	验证应用或 API 只允许明确支持的 HTTP 方法，包括需要用于预检请求的 OPTIONS；所有未使用的方法都必须禁用。	3
4.1.5	验证对高度敏感或经过多个系统的请求或业务操作使用逐消息数字签名，在传输保护的基础上提供额外保证。	3

V4.2 HTTP 消息结构验证

本节说明应如何验证 HTTP 消息的结构和头字段，以防止请求走私、响应拆分、头注入，以及由过长 HTTP 消息导致的拒绝服务等攻击。

这些要求适用于一般 HTTP 消息处理和生成，但在不同 HTTP 版本之间转换 HTTP 消息时尤其重要。

#	描述	级别
4.2.1	验证所有应用组件（包括负载均衡器、防火墙和应用服务器）都按照相应 HTTP 版本的规则确定传入消息的边界，以防范 HTTP 请求走私。对于 HTTP/1.x，如果消息中含有 Transfer-Encoding 头，则必须按照 RFC 2616 忽略 Content-Length。对于 HTTP/2 或 HTTP/3，如果消息中含有 Content-Length，接收方必须确认其值与 DATA 帧的实际长度一致。	2
4.2.2	验证应用生成 HTTP 消息时，Content-Length 头所声明的长度与 HTTP 分帧机制确定的实际内容长度一致，以防范请求走私。	3
4.2.3	验证应用既不发送也不接受含有连接专用头字段（例如 Transfer-Encoding）的 HTTP/2 或 HTTP/3 消息，以防范响应拆分和头注入。	3
4.2.4	验证应用只接受头字段名称和值中均不含 CR (\escape{r})、LF (\escape{n}) 或 CRLF (\escape{r}\escape{n}) 序列的 HTTP/2 和 HTTP/3 请求，以防范头注入。	3
4.2.5	验证应用的后端或前端在构造并发送请求时，会通过验证、净化或其他机制限制 URI（例如 API 调用地址）和 HTTP 请求头字段（例如 Authorization 或 Cookie）的长度，避免接收方因内容过长而拒绝请求。过长的请求可能造成拒绝服务；例如，发送过长的请求（如过长的 Cookie 头字段）可能导致服务器始终返回错误状态。	3

V4.3 GraphQL

GraphQL 越来越多地用于构建数据密集型客户端，使客户端无需与多个后端服务紧密耦合。本节介绍使用 GraphQL 时需要注意的安全问题。

#	描述	级别
4.3.1	验证应用通过查询允许列表、查询深度限制、数量限制或查询成本分析，防止复杂的嵌套查询耗尽 GraphQL 服务或数据层资源并造成拒绝服务（DoS）。	2
4.3.2	验证生产环境已经禁用 GraphQL 内省查询；如果该 GraphQL API 本来就需要提供给外部使用方，则可以例外。	2

V4.4 WebSocket

WebSocket 是一种通信协议，可在单个 TCP 连接上提供同时双向通信通道。它于 2011 年由 IETF 以 RFC 6455 标准化；虽然它被设计为通过 HTTP 端口 443 和 80 工作，但它不同于 HTTP。

本节提供关键安全要求，用于防止专门利用这种实时通信通道的通信安全和会话管理相关攻击。

#	描述	级别
4.4.1	验证所有 WebSocket 连接都使用基于 TLS 的 WebSocket (WSS)。	1
4.4.2	验证初次进行 HTTP WebSocket 握手时，会对照应用允许的源列表检查 Origin 头字段。	2
4.4.3	验证无法沿用应用原有会话管理机制时，WebSocket 使用专用的会话令牌，并且这些令牌满足相关的会话管理安全要求。	2
4.4.4	验证把现有 HTTPS 会话升级为 WebSocket 通道时，专用 WebSocket 会话令牌最初是通过已经认证的 HTTPS 会话获取或验证的。	2

参考资料

更多信息，另见：

- OWASP REST Security Cheat Sheet
- 来自 graphql.org 和 Apollo 的 GraphQL 授权资源。
- OWASP Web Security Testing Guide: GraphQL Testing
- OWASP Web Security Testing Guide: Testing WebSockets

V5 文件处理

控制目标

应用在接收、存储和提供文件时，会面临拒绝服务、未授权访问和存储空间耗尽等风险。本章说明如何应对这些风险。

V5.1 文件处理文档

本节要求记录应用可以接收哪些文件，以及这些文件应具备哪些特征。这些信息是开发和验证文件安全检查的前提。

#	描述	级别
5.1.1	验证文档针对每项上传功能，规定了允许的文件类型、预期的文件扩展名和大小上限（包括解压或解包后的大小）。文档还必须说明如何确保最终用户能够安全地下载和处理文件，例如应用检测到恶意文件时应如何处理。	2

V5.2 文件上传和内容

上传功能是应用接收不可信文件的主要入口。本节要求应用限制文件的大小、数量和内容，避免上传文件危害应用。

#	描述	级别
5.2.1	验证应用只接受大小在其处理能力范围内的文件，且处理这些文件不会造成性能下降或拒绝服务攻击。	1
5.2.2	验证应用接收文件时，会检查扩展名是否在允许范围内，并确认文件内容与扩展名表示的类型一致。此要求既适用于单独上传的文件，也适用于 ZIP 等归档文件中的每个文件。检查方法包括但不限于校验文件开头的“魔数”、重新编码图片，以及使用专用库验证文件内容。L1 可以只检查用于作出特定业务或安全决策的文件；L2 及以上必须检查所有接收的文件。	1
5.2.3	验证应用在解压或解包 ZIP、GZ、DOCX、ODT 等文件之前，会检查解压后的总大小和文件数量是否超过规定上限。	2
5.2.4	验证应用限制每个用户可存储文件的总大小和文件数量，防止单个用户通过上传过多或过大的文件耗尽存储空间。	3
5.2.5	验证应用拒绝包含符号链接的压缩文件。业务确实需要支持符号链接时，必须通过允许列表严格限制符号链接可以指向的文件。	3
5.2.6	验证应用拒绝像素尺寸超过最大允许值的上传图片，以防止像素洪泛攻击。	3

V5.3 文件存储

本节要求应用防止上传的文件被当作代码执行，识别其中的危险内容，并避免由不可信数据决定文件的存储位置。

#	描述	级别
5.3.1	验证由不可信输入上传或生成的文件，即使存放在可公开访问的目录中，也不会通过在 HTTP 直接访问时被服务端当作程序代码执行。	1

#	描述	级别
5.3.2	验证应用执行文件操作时，使用内部生成的数据或可信数据来构造文件路径，而不是直接使用用户提交的文件名。确实需要使用用户提供的文件名或文件元数据时，必须进行严格验证和净化，以防范路径遍历、本地或远程文件包含（LFI、RFI）以及服务端请求伪造（SSRF）。	1
5.3.3	验证服务端在解压文件等处理过程中忽略用户提供的路径信息，以防范 Zip Slip 等漏洞。	3

V5.4 文件下载

本节说明提供文件下载时如何防范路径遍历和注入攻击，并避免向用户提供含有恶意内容的文件。

#	描述	级别
5.4.1	验证应用会验证或忽略用户提交的文件名，包括通过 JSON、JSONP 或 URL 参数提交的文件名；下载响应中的实际文件名由应用在 Content-Disposition 头中明确指定。	2
5.4.2	验证对外提供的文件名（例如 HTTP 响应头字段或电子邮件附件中的文件名）经过编码或净化（例如遵循 RFC 6266），以保持文档结构并防止注入攻击。	2
5.4.3	验证来自不可信来源的文件经过防病毒软件扫描，避免向用户提供已知的恶意内容。	2

参考资料

更多信息，另见：

- OWASP File Upload Cheat Sheet
- Example of using symlinks for arbitrary file read
- Explanation of “Magic Bytes” from Wikipedia

V6 认证

控制目标

认证是确认个人或设备真实身份的过程。认证机制需要核实主体声称的身份，防止他人冒充，并防止密码被恢复或截获。

NIST SP 800-63 是一套以研究证据为基础的现代数字身份标准，对世界各地的组织都有参考价值，尤其适用于美国机构及其合作组织。

本章许多要求源自该标准的第二部分，即 NIST SP 800-63B《数字身份指南——认证和生命周期管理》。不过，本章只重点讨论常见威胁和经常遭到利用的认证弱点，并不覆盖该标准的全部内容。如需完整遵循 NIST SP 800-63，请直接查阅该标准。

此外，NIST SP 800-63 的术语有时不同；为提高清晰度，本章通常使用更容易理解的常用术语。

较为先进的应用通常能够根据各种风险因素调整所需的认证步骤。该特性在“授权”章节中介绍，因为这些机制也需要纳入授权决策。

V6.1 认证文档

本节规定应用需要维护哪些认证文档。清楚的文档是正确配置和评估认证控制的前提。

#	描述	级别
6.1.1	验证应用文档说明了如何利用速率限制、反自动化和自适应响应等控制，防范凭据填充和密码暴力破解。文档必须写明这些控制的配置方式，以及如何避免攻击者恶意触发账号锁定。	1
6.1.2	验证文档记录了与应用具体情境相关的禁用词表，防止这些词被用于密码。词表可以包括组织名称、产品名称、系统标识符、项目代号、部门或角色名称，以及这些名称的常见变体。	2
6.1.3	验证应用存在多条认证路径时，文档列出了所有路径，并规定了各条路径必须一致采用的安全控制和认证强度。	2

V6.2 密码安全

NIST SP 800-63把用户需要记住的认证信息统称为“记忆秘密”，其中包括密码、口令短语、PIN、解锁图案，以及选出正确的小猫或其他图像元素等方式。这些信息属于“用户知道的内容”，经常用于单因素认证。

因此，本节包含确保密码被安全创建和处理的要求。大多数要求属于 L1，因为它们在该等级最重要。从 L2 开始，需要多因素认证机制，而密码可能只是其中一个因素。

本节要求主要与 NIST 指南的 § 5.1.1.2 相关。

#	描述	级别
6.2.1	验证用户设置的密码至少包含 8 个字符；强烈建议把最短长度提高到 15 个字符。	1
6.2.2	验证用户可以修改自己的密码。	1
6.2.3	验证密码修改功能要求用户提供当前密码和新密码。	1
6.2.4	验证用户注册账号或修改密码时，应用会把新密码与常见密码列表进行比对。列表至少应包含符合应用密码策略（例如最短长度）的最常见的 3000 个密码。	1
6.2.5	验证应用允许用户自由组合密码中的字符，不以复杂度规则限制字符类型。不得强制密码必须包含一定数量的大写字母、小写字母、数字或特殊字符。	1

#	描述	级别
6.2.6	验证密码输入框使用 <code>type=password</code> 遮蔽输入内容。应用可以允许用户临时显示完整密码，也可以只短暂显示最后输入的字符。	1
6.2.7	验证密码输入框允许粘贴，并支持浏览器的密码辅助功能和第三方密码管理器。	1
6.2.8	验证应用完全按照用户提交的原始内容校验密码，不得截断密码，也不得改变字母大小写或进行其他转换。	1
6.2.9	验证应用支持长度至少为 64 个字符的密码。	2
6.2.10	验证用户密码在确认已经泄露或用户主动更换之前始终有效。应用不得要求定期更换凭据。	2
6.2.11	验证应用使用文档中规定的与具体情境相关的禁用词表，阻止用户设置容易猜中的密码。	2
6.2.12	验证用户注册账号或修改密码时，应用会把新密码与已知泄露密码库进行比对。	2

V6.3 通用认证安全

本节规定认证机制普遍需要满足的安全要求，并区分不同验证等级。L2应用必须强制使用多因素认证（MFA）。L3应用必须采用基于硬件的认证，且认证必须在经过证明的可信执行环境（TEE）中执行。例如，可以使用与设备绑定的通行密钥、达到 eIDAS 高保证等级（LoA High）的认证器、达到 NIST 认证器保证等级 3（AAL3）的认证器，或同等强度的机制。

这些 MFA 要求较为严格，但提高认证门槛对保护用户十分重要。如果尝试放宽这些要求，应制定明确方案，说明如何缓解认证风险，并考虑 NIST 指南及相关研究。

请注意，在发布时，NIST SP 800-63 将电子邮件视为不可接受的认证机制（归档副本）。

本节要求涉及 NIST 指南中的多个章节，包括：§ 4.2.1、§ 4.3.1、§ 5.2.2 和 § 6.1.2。

#	描述	级别
6.3.1	验证应用按照安全文档落实了防范凭据填充和密码暴力破解的控制措施。	1
6.3.2	验证应用不存在仍可使用的默认账号，例如 <code>root</code> 、 <code>admin</code> 或 <code>sa</code> ；如有此类账号，必须将其禁用。	1
6.3.3	验证用户必须通过多因素认证，或者组合使用多个单因素认证机制，才能访问应用。对于 L3，至少一个因素必须采用基于硬件的认证机制，能够抵抗网络钓鱼造成的凭据泄露和身份冒充；该机制还必须要求用户主动操作（例如按下 FIDO 硬件密钥或手机上的按钮），以确认用户确实有意登录。若要放宽其中任何一项要求，必须完整记录理由，并配套一组充分的补偿控制。	2
6.3.4	验证应用包含多条认证路径时，所有路径都已记录，并且各条路径一致地执行安全控制和认证强度要求。	2

#	描述	级别
6.3.5	验证应用会把可疑的认证尝试通知用户，无论尝试成功还是失败。可疑情形可能包括从异常地点或客户端发起认证、多因素认证中只有部分因素通过、账号长期未使用后出现认证尝试，以及连续多次失败后突然认证成功。	3
6.3.6	验证应用不把电子邮件用作单因素认证，也不把它用作多因素认证中的一个因素。	3
6.3.7	验证认证信息发生变更后，应用会通知用户，例如重置凭据，或者修改用户名、电子邮件地址。	3
6.3.8	验证攻击者无法根据失败的认证结果判断某个用户是否存在，包括无法利用错误消息、HTTP 状态码或响应时间差异进行推断。账号注册和忘记密码功能也必须提供同等保护。	3

V6.4 认证因素生命周期和恢复

认证因素可以包括密码、软令牌、硬件令牌和生物识别设备。安全处理这些机制的生命周期对应用安全至关重要，本节包含相关要求。

本节要求主要与 NIST 指南的 § 5.1.1.2 或 § 6.1.2.3 相关。

#	描述	级别
6.4.1	验证系统生成的初始密码或激活码采用安全的随机方式生成，符合现行密码策略，并在较短时间后或首次使用后失效。不得允许这些初始秘密成为长期密码。	1
6.4.2	验证应用没有使用密码提示，也没有使用所谓“秘密问题”等基于知识的认证方式。	1
6.4.3	验证应用提供了安全的忘记密码重置流程，并且该流程不会绕过已经启用的多因素认证。	2
6.4.4	验证用户丢失多因素认证中的某个因素时，应用会采用与注册该因素时同等强度的身份核验流程。	2
6.4.5	验证应用会在认证机制到期前留出足够时间，向用户发送续期说明，使用户能够在原机制失效前完成续期；必要时还应自动提醒用户。	3
6.4.6	验证管理员可以为用户发起密码重置，但不能替用户指定或修改密码，从而避免管理员知晓用户密码。	3

V6.5 通用多因素认证要求

本节提供适用于多种不同多因素认证方法的一般指导。

这些机制包括：

- 查表型秘密
- 基于时间的一次性密码（TOTP）

• 带外机制

查表型秘密 (look-up secret) 是预先生成的一组秘密代码，例如交易授权码 (TAN)、社交媒体恢复码，或由随机值组成的代码表。这些代码通常难以记忆，需要保存在某种介质中，因此属于“用户拥有的内容”。

基于时间的一次性密码 (TOTP) 由硬件令牌或软件令牌生成，显示会随时间变化的伪随机一次性密码，因此也属于“用户拥有的内容”。多因素 TOTP 与单因素 TOTP 类似，但生成最终的一次性密码 (OTP) 前，还要求输入有效 PIN、通过生物识别解锁、插入 USB 设备或完成 NFC 配对，或输入某个附加值 (例如使用交易签名计算器时)。

带外机制的细节将在下一节提供。

这些章节中的要求主要与 NIST 指南的 § 5.1.2、§ 5.1.3、§ 5.1.4.2、§ 5.1.5.2、§ 5.2.1 和 § 5.2.3 相关。

#	描述	级别
6.5.1	验证每个查表型秘密、带外认证请求或验证码，以及基于时间的一次性密码 (TOTP)，最多只能成功使用一次。	2
6.5.2	验证存储在应用后端的查表型秘密如果少于 112 位熵 (约相当于 19 个随机字母数字字符或 34 个随机数字)，会使用获批准的密码存储哈希算法和 32 位随机盐进行哈希。秘密达到或超过 112 位熵时，可以改用普通的密码学哈希函数。	2
6.5.3	验证查表型秘密、带外验证码和 TOTP 种子都由密码学安全伪随机数生成器 (CSPRNG) 生成，避免生成可预测的值。	2
6.5.4	验证查表型秘密和带外验证码至少包含 20 位熵，通常 4 个随机字母数字字符或 6 个随机数字即可满足要求。	2
6.5.5	验证应用为带外认证请求、验证码或令牌以及 TOTP 设置了明确的有效期。带外认证请求的有效期最长为 10 分钟，TOTP 的有效期最长为 30 秒。	2
6.5.6	验证包括实体设备在内的任何认证因素，在被盗或丢失后都可以撤销。	3
6.5.7	验证生物识别只能作为辅助因素，必须与“用户拥有的内容”或“用户知道的内容”配合使用。	3
6.5.8	验证 TOTP 使用可信服务提供的时间源进行校验，不采用不可信来源或客户端提供的时间。	3

V6.6 带外认证机制

带外认证通常由认证服务器通过另一条安全通道与用户设备通信，例如向手机发送推送通知。由于认证依赖用户持有该设备，因此属于“用户拥有的内容”。

不得使用电子邮件、VoIP 等不安全的带外认证方式。NIST 目前把通过 PSTN 或 SMS 进行的认证列为“受限”认证机制，应用应逐步停用这些方式，改用 TOTP、密码学认证或类似机制。NIST SP 800-63B § 5.1.3.3 指出，如果确实必须保留电话或 SMS 带外认证，就应处理更换设备、更换 SIM 卡、号码携转和其他异常行为带来的风险。本 ASVS 小节虽然没有把这些措施列为强制要求，但敏感的 L2 应用和所有 L3 应用如果没有采取相应措施，应视为存在重大风险。

请注意，NIST 最近还提供了不鼓励使用推送通知的指导。虽然本 ASVS 小节没有这样规定，但了解“推送轰炸”风险很重要。

#	描述	级别
6.6.1	验证应用只有同时满足以下条件时，才允许通过公共交换电话网络（PSTN）的语音电话或 SMS 发送一次性密码（OTP）：电话号码已经预先验证；同时提供 TOTP 等更强的替代方式；并且已经向用户说明其中的安全风险。L3 应用不得提供电话或 SMS 认证。	2
6.6.2	验证每个带外认证请求、验证码或令牌都与产生它的那一次认证请求绑定，不能挪用于之前或之后的其他请求。	2
6.6.3	验证基于验证码的带外认证采用速率限制来防范暴力破解。也可以考虑使用至少包含 64 位熵的验证码。	2
6.6.4	验证使用推送通知进行多因素认证时，应用通过速率限制防范“推送轰炸”。数字匹配也可以降低这类风险。	3

V6.7 密码学认证机制

密码学认证机制包括智能卡和 FIDO 安全密钥等。用户需要把密码学设备连接或配对到计算机，才能完成认证。认证服务器向设备或软件发送一个挑战值（nonce），设备或软件再使用安全存储的密码学密钥计算响应。本节说明这类机制的实现要求；算法选择方面的要求见“密码学”章节。

如果密码学认证使用共享密钥或秘密密钥，这些密钥应使用与其他系统秘密相同的机制进行存储，具体见“配置”章节中的“秘密管理”小节。

本节要求主要与 NIST 指南的 § 5.1.7.2 相关。

#	描述	级别
6.7.1	验证用于核验密码学认证断言的证书采用防篡改方式存储。	3
6.7.2	验证挑战值（nonce）至少为 64 位，并且在统计意义上不会重复，或者保证在密码学设备的整个生命周期内不重复。	3

V6.8 使用身份提供方进行认证

身份提供方（IdP）为用户提供联合身份。用户通常会在多个 IdP 中拥有多个身份，例如使用 Azure AD、Okta、Ping Identity 或 Google 的企业身份，或使用 Facebook、Twitter、Google、WeChat 等个人身份。这些只是常见选择，并不代表对这些公司或服务的认可，而是提醒开发人员考虑一个现实：许多用户已经拥有多个既有身份。组织应根据 IdP 身份核验强度的风险评估，考虑是否与现有用户身份集成。例如，政府组织不太可能接受社交媒体身份作为敏感系统的登录方式，因为这类身份很容易伪造或仅供一次性使用；而移动游戏公司则很可能需要与主要社交媒体平台集成，以扩大活跃玩家群体。

安全使用外部身份提供方需要谨慎配置和验证，以防止身份冒充或伪造断言。本节提供处理这些风险的要求。

#	描述	级别
6.8.1	验证应用支持多个身份提供方（IdP）时，用户身份不能通过另一个受支持的 IdP 被冒充，例如通过使用相同的用户标识符进行冒充。通常，应用应把 IdP 标识作为命名空间，并将其与用户在该 IdP 中的标识组合起来，用于注册和识别用户。	2
6.8.2	验证始终检查认证断言（例如 JWT 或 SAML 断言）是否带有数字签名，以及签名是否完整有效，并拒绝任何未签名或签名无效的断言。	2
6.8.3	验证每条 SAML 断言在有效期内只处理一次，不能重复使用，从而防范重放攻击。	2
6.8.4	验证应用使用外部身份提供方（IdP）时，如果某项功能要求特定的认证强度、认证方式或认证时效性，就会根据 IdP 返回的信息确认要求是否满足。例如，使用 OIDC 时，可以检查 ID Token 中的 acr、amr 和 auth_time 声明（如有）。如果 IdP 不提供这些信息，应用必须有文档化的回退方案，并在方案中假定使用了最低强度的认证机制，例如使用用户名和密码的单因素认证。	2

参考资料

更多信息，另见：

- NIST SP 800-63 - Digital Identity Guidelines
- NIST SP 800-63B - Authentication and Lifecycle Management
- NIST SP 800-63 FAQ
- OWASP Web Security Testing Guide: Testing for Authentication
- OWASP Password Storage Cheat Sheet
- OWASP Forgot Password Cheat Sheet
- OWASP Choosing and Using Security Questions Cheat Sheet
- CISA Guidance on “Number Matching”
- Details on the FIDO Alliance

V7 会话管理

控制目标

即使底层使用 HTTP 这类无状态协议，会话管理机制也能让应用在一段时间内识别并关联同一用户或设备的多次交互。现代应用可能同时使用多种会话令牌，每种令牌的用途和特性各不相同。安全的会话管理系统必须防止攻击者窃取、使用或以其他方式滥用他人的会话，并满足以下总体要求：

- 会话对每个个体都是唯一的，不能被猜测或共享。
- 当不再需要会话时，会话会失效；在不活动期间，会话会超时。

本章许多要求都与选定的 NIST SP 800-63 Digital Identity Guidelines 控制相关，重点关注常见威胁和常被利用的认证弱点。

请注意，某些会话管理机制的具体实现细节要求可在其他章节中找到：

- HTTP Cookie 是保护会话令牌的常见机制。Cookie 的具体安全要求位于“Web 前端安全”章节。
- 自包含令牌经常用于维护会话。具体安全要求位于“自包含令牌”章节。

V7.1 会话管理文档

没有一种会话管理方案适合所有应用，也无法为所有场景规定统一的边界和限制。因此，在实现和测试会话管理之前，必须先分析相关风险并记录安全决策，确保最终方案符合应用的实际需求。

无论选择有状态还是“无状态”会话机制，分析都必须完整并形成文档，以证明所选方案能够满足所有相关安全要求。还应考虑与任何正在使用的单点登录（SSO）机制之间的交互。

#	描述	级别
7.1.1	验证用户会话的不活动超时时间和会话绝对最长生命周期已记录在文档中，并且这些限制与其他控制配合使用时是适当的。如果相关设置偏离 NIST SP 800-63B 的重新认证要求，文档必须说明理由。	2
7.1.2	验证文档规定了每个账号允许同时存在的会话数量，以及达到上限时应用应如何处理。	2
7.1.3	验证文档列出了联合身份体系（例如 SSO 系统）中所有负责创建和管理用户会话的系统，并说明各系统如何协调会话的生命周期、终止操作和重新认证条件。	2

V7.2 基础会话管理安全

本节规定安全生成和验证会话令牌的基本要求。

#	描述	级别
7.2.1	验证应用使用可信后端服务执行所有会话令牌验证。	1
7.2.2	验证应用使用动态生成的自包含令牌或引用令牌来管理会话，而不是使用静态的 API 秘密和密钥。	1
7.2.3	验证应用使用引用令牌表示用户会话时，每个令牌都具有唯一性，由密码学安全伪随机数生成器（CSPRNG）生成，并且至少包含 128 位熵。	1
7.2.4	验证应用在用户认证（包括重新认证）时生成新的会话令牌，并使当前会话令牌失效。	1

V7.3 会话超时

会话超时机制用于尽量缩小会话劫持和其他会话滥用的机会窗口。超时设置必须满足已记录的安全决策。

#	描述	级别
7.3.1	验证应用设置了会话不活动超时，并按照风险分析和已记录的安全决策，在超时后强制用户重新认证。	2
7.3.2	验证应用设置了会话绝对最长生命周期，并按照风险分析和已记录的安全决策，在达到上限后强制用户重新认证。	2

V7.4 会话终止

会话可以由应用自身终止；如果应用把会话管理交给了 SSO 提供方，也可以由 SSO 提供方终止。评估本节要求时，应判断是否需要把 SSO 提供方纳入验证范围，因为部分会话控制可能由它负责。

会话终止后应要求用户重新认证，且终止操作应在应用、联合登录系统（如有）和所有依赖方生效。

对于有状态会话机制，终止通常涉及在后端使会话失效。对于自包含令牌，需要额外措施来撤销或阻止这些令牌，否则它们可能在过期前仍然有效。

#	描述	级别
7.4.1	验证会话被注销或过期等事件终止后，应用不再接受该会话。对于引用令牌或有状态会话，这意味着在应用后端使相应会话数据失效。使用自包含令牌的应用需要采用相应方案，例如维护已终止令牌列表、拒绝某个用户在指定时间之前签发的令牌，或轮换该用户的签名密钥。	1
7.4.2	验证用户账号被禁用或删除时（例如员工离职），应用会终止该账号的所有活动会话。	1
7.4.3	验证用户成功更改或移除任何认证因素后，应用允许其终止所有其他活动会话。认证因素的变更包括通过重置或恢复流程修改密码，以及更新已有的 MFA 设置。	2
7.4.4	验证所有需要认证的页面都在醒目、易用的位置提供注销功能。	2
7.4.5	验证应用管理员能够终止单个用户或所有用户的活动会话。	2

V7.5 防范会话滥用

本节提供相关要求，用于降低活动会话遭到劫持或滥用的风险。这类攻击会利用用户已登录的会话及其权限，例如通过执行恶意内容，强制已认证用户的浏览器借助其会话执行某项操作。

请注意，在考虑本节要求时，应同时参考“认证”章节中针对不同等级的指导。

#	描述	级别
7.5.1	验证应用在允许用户修改可能影响认证的敏感账号信息之前，要求用户完成完整的重新认证。此类信息包括电子邮件地址、电话号码、MFA 配置，以及账号恢复所需的其他信息。	2

#	描述	级别
7.5.2	验证用户可以查看自己的任一或全部活动会话，并能在使用至少一个认证因素重新认证后终止这些会话。	2
7.5.3	验证应用在执行高度敏感的交易或其他业务操作之前，要求用户使用至少一个认证因素再次认证，或者完成二次验证。	3

V7.6 联合重新认证

本节与编写依赖方（RP）或身份提供方（IdP）代码的人员相关。这些要求源自 NIST SP 800-63C 中关于联合与断言的内容。

#	描述	级别
7.6.1	验证依赖方（RP）和身份提供方（IdP）的会话生命周期及终止方式符合文档规定，并在需要时强制重新认证。例如，当距离上一次 IdP 认证达到规定的最长间隔时，应要求用户重新认证。	2
7.6.2	验证创建会话必须得到用户同意或由用户明确触发，防止应用在没有用户参与的情况下自动创建新会话。	2

参考资料

更多信息，另见：

- OWASP Web Security Testing Guide: Session Management Testing
- OWASP Session Management Cheat Sheet

V8 授权

控制目标

授权的作用是确保只有获得许可的使用方（包括用户、服务器和其他客户端）才能访问资源。为了落实最小权限原则（POLP），应用必须满足以下总体要求：

- 记录授权规则，包括决策因素和环境上下文。
- 使用方只能访问其既定权限所允许的资源。

V8.1 授权文档

完整的授权文档有助于确保各处采用一致、可审计且符合组织策略的安全决策。文档应让开发人员、管理员和测试人员都能清楚理解并落实授权要求，从而降低未授权访问的风险。

#	描述	级别
8.1.1	验证授权文档规定了如何根据使用方的权限和资源属性，限制其可以使用哪些功能以及可以访问哪些具体数据。	1
8.1.2	验证授权文档规定了如何根据使用方的权限和资源属性，限制其可以读取或写入哪些字段。这些规则可能还取决于相关数据对象的其他属性，例如对象当前的状态。	2
8.1.3	验证应用文档规定了用于作出安全决策的环境和上下文属性（包括但不限于时间、用户位置、IP 地址或设备），这些决策包括与认证和授权有关的决策。	3
8.1.4	验证认证和授权文档说明了除功能级、数据级和字段级权限之外，环境和上下文因素如何参与安全决策。文档应列出需要评估的属性、风险阈值，以及不同评估结果对应的处理方式，例如允许、质询、拒绝或提升认证强度。	3

V8.2 通用授权设计

应用应在功能、数据和字段三个层面实施细粒度授权，确保使用方只能访问明确授予其权限的内容。

#	描述	级别
8.2.1	验证应用只允许拥有明确权限的使用方访问相应功能。	1
8.2.2	验证应用只允许拥有明确权限的使用方访问相应的数据项，以防范不安全直接对象引用 (IDOR) 和对象级授权失效 (BOLA)。	1
8.2.3	验证应用只允许拥有明确权限的使用方访问相应字段，以防范对象属性级授权失效 (BOPLA)。	2
8.2.4	验证应用按照文档规定，根据使用方的环境和上下文属性（例如时间、位置、IP 地址或设备）实施自适应安全控制，并将结果用于认证和授权决策。使用方尝试创建新会话时和现有会话存续期间都必须执行这些控制。	3

V8.3 操作级授权

授权变更在应用架构的适当层级立即生效，对防止未授权操作至关重要，在动态环境中尤其如此。

#	描述	级别
8.3.1	验证应用在可信服务层执行授权规则，不依赖客户端 JavaScript 等可能被不可信使用方操纵的控制。	1
8.3.2	验证授权决策所依据的值发生变化时，这些变更立即生效。受自包含令牌等机制限制而无法立即生效时，必须采取补偿措施：在已经失去权限的使用方继续执行操作时发出告警，并回滚其变更。注意，这种补偿措施无法防止信息已经被读取或泄露。	3
8.3.3	验证应用根据最初发起请求的主体（例如最终用户）的权限决定是否允许访问对象，而不是采用代其调用的中间服务的权限。例如，用户携带自包含令牌调用第一个 Web 服务，该服务再向第二个服务请求数据时，第二个服务应根据用户令牌作出授权决定，而不能采用第一个服务的机器间令牌所具有的权限。	3

V8.4 其他授权考虑

授权方面的其他考虑，尤其是针对管理接口和多租户环境的考虑，有助于防止未授权访问。

#	描述	级别
8.4.1	验证多租户应用实施了租户隔离控制，确保使用方的操作不会影响其无权与之交互的租户。	2
8.4.2	验证管理接口采用多层安全措施，包括持续验证使用方身份、评估设备安全状态和分析上下文风险。网络位置或可信端点可以作为降低风险的因素，但不得成为授权访问的唯一依据。	3

参考资料

更多信息，另见：

- OWASP Web Security Testing Guide: Authorization
- OWASP Authorization Cheat Sheet

V9 自包含令牌

控制目标

2012年发布的 OAuth 2.0 原始规范 RFC 6749 提到了自包含令牌的概念。所谓自包含令牌，是指令牌本身携带数据或声明，接收服务可以直接依据这些内容作出安全决策。与之不同的是，有些令牌只包含一个标识符，接收服务必须使用该标识符在本地查询实际数据。常见的自包含令牌包括 JSON Web Token (JWT) 和 SAML 断言。

如今，自包含令牌已经广泛使用，甚至用于 OAuth 和 OIDC 之外的场景。这一机制的安全性依赖于验证令牌完整性，并确认令牌在特定上下文中有效的能力。这一过程存在许多陷阱，本章详细说明了应用应采取的防范机制。

V9.1 令牌来源和完整性

本节要求应用确认令牌由可信方签发，并验证令牌内容没有遭到篡改。

#	描述	级别
9.1.1	验证应用在接受自包含令牌中的任何内容之前，会使用数字签名或消息认证码（MAC）验证令牌完整性，确认其内容没有遭到篡改。	1
9.1.2	验证创建和校验自包含令牌时，只允许使用当前场景预先批准的算法。算法允许列表不得包含 None，并且最好只允许使用对称算法或非对称算法中的一类。如果必须同时支持这两类算法，则必须采取额外措施防止密钥混淆。	1
9.1.3	验证用于校验自包含令牌的密钥材料，来自预先为相应令牌签发方配置的可信来源，以防止攻击者指定不可信的来源和密钥。对于 JWT 和其他 JWS 结构，必须对照可信来源允许列表检查 jku、x5u、jwk 等头字段。	1

V9.2 令牌内容

服务根据自包含令牌中的内容作出安全决策前，必须确认该令牌仍在有效期内、允许由当前服务使用，并且令牌类型符合当前用途。这样可以防止一个服务接受原本签发给另一个服务的令牌，也可以防止混用同一签发方签发的不同类型令牌。

OAuth 和 OIDC 的相关具体要求见专门章节。

#	描述	级别
9.2.1	验证令牌中规定了有效时间范围时，接收方只在校验时间位于该范围内时，才接受令牌及其中的内容。对于 JWT，必须检查 nbf 和 exp 声明。	1
9.2.2	验证服务在接受令牌内容之前，会确认令牌类型正确，并且符合当前使用目的。例如，只有访问令牌可以用于授权决策，只有 ID Token 可以用于证明用户已经完成认证。	2
9.2.3	验证服务在接受令牌前，会确认该令牌的预期接收方（受众）包含本服务。对于 JWT，可以通过对照服务中定义的允许列表验证 aud 声明来实现。	2
9.2.4	验证签发方使用同一私钥为不同受众签发令牌时，签发的令牌包含能够唯一标识预期受众的受众限制，以防止令牌被非预期受众重复使用。受众标识符采用动态分配方式时，签发方必须验证这些受众，确保不会导致受众冒充。	2

参考资料

更多信息，另见：

- OWASP JSON Web Token Cheat Sheet for Java（其中也包含实用的通用指导）

V10 OAuth 和 OIDC

控制目标

OAuth 2.0（本章简称 OAuth）是用于委派授权的行业标准框架。用户授权后，客户端应用便可以代表用户访问 API 等服务端资源。

OAuth 本身并不是用户认证协议。OpenID Connect（OIDC）在 OAuth 之上增加了身份层，提供标准化用户信息、单点登录（SSO）和会话管理等功能。由于 OIDC 建立在 OAuth 之上，本章针对 OAuth 的要求也适用于 OIDC。

OAuth 定义了以下角色：

- OAuth 客户端是尝试获取服务器资源访问权限的应用（例如使用已签发的访问令牌调用 API）。OAuth 客户端通常是服务端应用。
 - 机密客户端能够妥善保管用于向授权服务器证明自身身份的凭据。
 - 公共客户端无法安全保管这类凭据。因此，它不能使用 `client_id` 和 `client_secret` 等参数证明自身身份，只能通过 `client_id` 标明自己是哪一个客户端。
- OAuth 资源服务器（RS）是向 OAuth 客户端暴露资源的服务器 API。
- OAuth 授权服务器（AS）是向 OAuth 客户端签发访问令牌的服务器应用。这些访问令牌允许 OAuth 客户端代表最终用户，或代表 OAuth 客户端自身，访问 RS 资源。AS 通常是独立应用，但在合适情况下也可以集成到合适的 RS 中。
- 资源所有者（RO）是最终用户，他们授权 OAuth 客户端代表自己获取对资源服务器上托管资源的有限访问权限。资源所有者通过与授权服务器交互来同意这种委派授权。

OIDC 定义了以下角色：

- 在 OIDC 中，接入 OIDC 登录并请求身份认证服务确认用户身份的应用称为“依赖方”（Relying Party, RP）。它在 OAuth 流程中同时承担客户端角色。
- OpenID Provider（OP）是能够认证最终用户并向 RP 提供 OIDC 声明的 OAuth AS。OP 可以是身份提供方（IdP），但在联合场景中，OP 和最终用户进行认证的身份提供方可能是不同的服务器应用。

OAuth 和 OIDC 最初面向第三方应用设计，如今也经常用于第一方应用。但在第一方场景中把它们用于认证和会话管理，会增加系统复杂度，并可能带来新的安全问题。

OAuth 和 OIDC 可用于多种类型的应用，但 ASVS 以及本章要求的重点是 Web 应用和 API。

OAuth 和 OIDC 建立在 Web 技术之上，因此其他章节的通用要求始终适用。不能脱离整份标准，孤立地理解或验证本章。

本章采用 OAuth 2.0 和 OIDC 当前的最佳实践，并与 <https://oauth.net/2/> 和 <https://openid.net/developers/specs/> 发布的规范保持一致。相关 RFC 即使已经成熟，仍会持续更新，因此，实施本章要求时，与最新版本保持一致十分重要。更多信息见参考资料。

鉴于该领域的复杂性，要构建安全的 OAuth 或 OIDC 解决方案，使用知名且符合行业标准的授权服务器并采用推荐的安全配置至关重要。

本章使用的术语与 OAuth RFC 和 OIDC 规范一致，但请注意，OIDC 术语只用于 OIDC 特定要求；其他情况下使用 OAuth 术语。

在 OAuth 和 OIDC 语境中，本章中的“令牌”指：

- 访问令牌只能由资源服务器（RS）使用。它可以是通过令牌内省进行验证的引用令牌，也可以是使用相应密钥材料进行验证的自包含令牌。
- 刷新令牌只能由签发该令牌的授权服务器接收并使用。
- OIDC ID Token 只能由触发相应授权流程的客户端接收并使用。

本章部分要求的等级取决于客户端属于机密客户端还是公共客户端。强客户端认证能够降低多种攻击风险，因此 L1 应用使用机密客户端时，可以适当放宽少数要求。

V10.1 通用 OAuth 和 OIDC 安全

本节覆盖适用于所有使用 OAuth 或 OIDC 的应用的通用架构要求。

#	描述	级别
10.1.1	验证令牌只发送给确实需要使用它的组件。例如，基于浏览器的 JavaScript 应用采用 BFF（Backend for Frontend）模式时，访问令牌和刷新令牌只能由后端访问。	2
10.1.2	验证客户端只接受由同一用户代理会话发起的同一次授权流程所产生的授权服务器返回值，例如授权码或 ID Token。为此，客户端生成的 code_verifier、state、OIDC nonce 等秘密值必须不可猜测、仅用于该次授权流程，并与客户端以及发起该流程的用户代理会话安全绑定。	2

V10.2 OAuth 客户端

本节详细说明 OAuth 客户端应用需要承担的安全责任。OAuth 客户端可以实现为运行在服务端的 Web 后端（通常充当 BFF）、用于服务间集成的后端服务，或者运行在浏览器中的单页应用（SPA，也称基于浏览器的应用）。

一般而言，后端客户端被视为机密客户端，前端客户端被视为公共客户端。不过，运行在最终用户设备上的原生应用在使用 OAuth 动态客户端注册时，也可以被视为机密客户端。

#	描述	级别
10.2.1	验证 OAuth 客户端使用授权码流程时，能够防范由浏览器请求伪造（通常称为 CSRF）触发的令牌请求。可以使用授权码交换证明密钥（PKCE），也可以检查授权请求中发送的 state 参数。	2
10.2.2	验证 OAuth 客户端需要与多个授权服务器交互时，能够防范授权服务器混淆攻击。例如，可以要求授权服务器返回 iss 参数，并在授权响应和令牌响应中检查该参数。	2
10.2.3	验证 OAuth 客户端向授权服务器发起请求时，只请求所需的 scope（或其他授权参数）。	3

V10.3 OAuth 资源服务器

在 ASVS 和本章语境中，资源服务器是一个 API。为提供安全访问，资源服务器必须：

- 根据令牌格式和相关协议规范验证访问令牌。例如，可以在本地验证 JWT，或者通过 OAuth 令牌内省端点向授权服务器查询令牌状态和属性。
- 如果令牌有效，则基于访问令牌中的信息和已授予权限执行授权决策。例如，资源服务器需要验证客户端（代表 RO 行事）是否被授权访问所请求的资源。

因此，以下 OAuth 和 OIDC 专用检查应在确认令牌有效之后、根据令牌内容作出授权决定之前执行。

#	描述	级别
10.3.1	验证资源服务器在接受访问令牌前，会确认该令牌的预期接收方（受众）包含本资源服务器。受众信息可以记录在结构化访问令牌中，例如 JWT 的 aud 声明；也可以通过令牌内省端点返回的信息确认。	2
10.3.2	验证资源服务器依据访问令牌中描述委派权限的声明作出授权决定。令牌含有 sub、scope、authorization_details 等声明时，必须把这些声明纳入决策。	2
10.3.3	验证访问控制决策需要从访问令牌（JWT 或相关令牌内省响应）识别唯一用户时，资源服务器只采用不会被重新分配给其他用户的声明。通常应组合使用 iss 和 sub。	2
10.3.4	验证资源服务器对认证强度、认证方式或认证时效性有特定要求时，会确认访问令牌满足这些条件。例如，可以分别检查 OIDC 的 acr、amr 和 auth_time 声明（如有）。	2
10.3.5	验证资源服务器要求使用发送方约束访问令牌，防止被盗访问令牌被使用，或访问令牌被未授权方重放。可以采用 OAuth 2.0 双向 TLS，或 OAuth 2.0 持有证明（DPoP）。	3

V10.4 OAuth 授权服务器

这些要求详细说明 OAuth 授权服务器的职责，包括 OpenID Provider。

客户端认证可以使用 self_signed_tls_client_auth，但必须满足 RFC 8705 第 2.2 节规定的前提条件。

#	描述	级别
10.4.1	验证授权服务器为每个客户端维护预先注册的重定向 URI 允许列表，并通过精确字符串比较来检查重定向 URI。	1
10.4.2	验证授权服务器在授权响应中返回授权码时，该授权码只能用于一次令牌请求。当第二次有效请求使用已用于签发访问令牌的授权码时，授权服务器必须拒绝该令牌请求，并撤销与该授权码相关的所有已签发令牌。	1
10.4.3	验证授权码的有效期足够短。L1 和 L2 应用最长为 10 分钟，L3 应用最长为 1 分钟。	1
10.4.4	验证授权服务器只为每个客户端启用其实际需要的授权类型。不得再使用 token（隐式流程）或 password（资源所有者密码凭据流程）授权。	1
10.4.5	验证授权服务器采取措施缓解针对公共客户端的刷新令牌重放攻击，优先使用发送方约束刷新令牌，即采用持有证明（DPoP）或使用双向 TLS（mTLS）的证书绑定访问令牌。L1 和 L2 应用可以使用刷新令牌轮换。采用轮换时，授权服务器必须在刷新令牌使用后使其失效；如果收到已使用且已失效的刷新令牌，必须撤销该授权对应的所有刷新令牌。	1
10.4.6	验证在使用授权码流程时，授权服务器强制采用 PKCE（Proof Key for Code Exchange）机制，以防范授权码截获攻击。授权服务器必须要求授权请求包含有效的 code_challenge，且不得接受值为 plain 的 code_challenge_method；对于令牌请求，必须要求验证 code_verifier 参数。	2
10.4.7	验证授权服务器允许未经认证的动态客户端注册时，采取措施降低恶意客户端风险。服务器必须验证已注册 URI 等客户端元数据，取得用户同意，并在处理不可信客户端的授权请求之前向用户发出警告。	2
10.4.8	验证刷新令牌设有绝对失效时间，即使同时采用滑动过期机制也不例外。	2
10.4.9	验证已授权用户可以通过授权服务器的用户界面撤销刷新令牌和引用访问令牌，以降低恶意客户端或令牌被盗带来的风险。	2
10.4.10	验证机密客户端向授权服务器发送后通道请求时会经过客户端认证，例如令牌请求、推送授权请求（PAR）和令牌撤销请求。	2
10.4.11	验证授权服务器配置只向 OAuth 客户端分配所需的 scope。	2
10.4.12	验证授权服务器只允许每个客户端使用其实际需要的 response_mode 值。可以把该值与预期值进行比较，也可以通过推送授权请求（PAR）或 JWT 安全授权请求（JAR）加以限制。	3
10.4.13	验证使用 code 授权类型时，始终同时采用推送授权请求（PAR）。	3
10.4.14	验证授权服务器只签发发送方约束（持有证明）访问令牌，采用基于双向 TLS（mTLS）的证书绑定访问令牌，或采用 DPoP 绑定访问令牌。	3
10.4.15	验证对于运行在服务器而非最终用户设备上的 OAuth 客户端，授权服务器必须确认 authorization_details 参数由该服务器上的客户端程序提供，并且未被用户篡改。可以通过强制使用推送授权请求（PAR）或 JWT 安全授权请求（JAR）来满足此要求。	3

#	描述	级别
10.4.16	验证客户端属于机密客户端，并且授权服务器强制使用基于公钥密码学、能够抵抗重放攻击的强客户端认证方式，例如双向 TLS (tls_client_auth、self_signed_tls_client_auth) 或私钥 JWT (private_key_jwt)。	3

V10.5 OIDC 客户端

本节适用于接入 OIDC 登录的应用，也就是 OIDC 规范所称的“依赖方”（RP）。由于这类应用在 OAuth 流程中同时承担客户端角色，因此“OAuth 客户端”小节中的要求也适用于它。

注意，“认证”章节中的“使用身份提供方进行认证”小节也包含相关通用要求。

#	描述	级别
10.5.1	验证接入 OIDC 登录的应用能够防范 ID Token 重放。例如，检查 ID Token 中的 nonce 是否与发送给 OpenID Provider 的认证请求中的 nonce 完全一致；在 OAuth 2.0 中，这类请求称为发送给授权服务器的授权请求。	2
10.5.2	验证客户端根据 ID Token 中不会被重新分配给其他用户的声明来唯一识别用户，通常使用相应身份提供方范围内的 sub 声明。	2
10.5.3	验证客户端能够阻止恶意授权服务器通过元数据冒充其他授权服务器。如果元数据中的签发方 URL 与客户端预先配置的预期 URL 不完全一致，客户端必须拒绝该元数据。	2
10.5.4	验证客户端检查 ID Token 的 aud 声明是否等于自身的 client_id，以确认该令牌确实以本客户端为受众。	2
10.5.5	验证使用 OIDC 后通道注销时，依赖方采取措施缓解注销流程中通过强制注销造成的拒绝服务和跨 JWT 混淆。客户端必须验证注销令牌的类型正确且值为 logout+jwt，包含成员名称正确的 event 声明，并且不包含 nonce 声明。此外，建议设置较短的有效期，例如 2 分钟。	2

V10.6 OpenID Provider

由于 OpenID Provider 充当 OAuth 授权服务器，“OAuth 授权服务器”小节中的要求同样适用。

注意，如果使用 ID Token 流程（而不是授权码流程），不会签发访问令牌，许多 OAuth AS 要求并不适用。

#	描述	级别
10.6.1	验证 OpenID Provider 只允许 response mode 使用 code、ciba、id_token 或 id_token code。应优先采用 code，而不是 id_token code (OIDC 混合流程)；不得使用 token 等任何隐式流程。	2
10.6.2	验证 OpenID Provider 采取措施缓解通过强制注销造成的拒绝服务，例如取得最终用户的明确确认，或者验证由依赖方发起的注销请求中提供的 id_token_hint 等参数 (如有)。	2

V10.7 同意管理

本节要求授权服务器正确取得并验证用户同意。否则，攻击者可能通过欺骗或社会工程手段，在用户不知情的情况下取得权限。

#	描述	级别
10.7.1	验证授权服务器确保用户同意每一次授权请求。无法确认客户端身份时，必须每次都明确询问用户是否同意。	2
10.7.2	验证授权服务器征求同意时，向用户提供充分、清晰的信息，说明其同意的授权内容。适用时，应说明所请求权限的性质（通常由 scope、资源服务器和富授权请求（RAR）详情决定）、获得授权的应用身份，以及授权的有效期。	2
10.7.3	验证用户可以查看、修改和撤销自己通过授权服务器作出的授权同意。	2

参考资料

关于 OAuth 的更多信息，请参见：

- oauth.net
- OWASP OAuth 2.0 Protocol Cheat Sheet

ASVS 中 OAuth 相关要求使用了以下已发布和草案状态的 RFC：

- RFC6749 The OAuth 2.0 Authorization Framework
- RFC6750 The OAuth 2.0 Authorization Framework: Bearer Token Usage
- RFC6819 OAuth 2.0 Threat Model and Security Considerations
- RFC7636 Proof Key for Code Exchange by OAuth Public Clients
- RFC7591 OAuth 2.0 Dynamic Client Registration Protocol
- RFC8628 OAuth 2.0 Device Authorization Grant
- RFC8707 Resource Indicators for OAuth 2.0
- RFC9068 JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens

- RFC9126 OAuth 2.0 Pushed Authorization Requests
- RFC9207 OAuth 2.0 Authorization Server Issuer Identification
- RFC9396 OAuth 2.0 Rich Authorization Requests
- RFC9449 OAuth 2.0 Demonstrating Proof of Possession (DPoP)
- RFC9700 Best Current Practice for OAuth 2.0 Security
- draft OAuth 2.0 for Browser-Based Applications
- draft The OAuth 2.1 Authorization Framework

关于 OpenID Connect 的更多信息，请参见：

- OpenID Connect Core 1.0
- FAPI 2.0 Security Profile

V11 密码学

控制目标

本章介绍应用密码技术时普遍适用的最佳实践，帮助读者理解基本原则，并推动应用采用更现代、更能适应未来变化的方案。主要目标包括：

- 建立安全可靠的密码技术体系，确保系统能够安全处理故障、应对不断变化的威胁，并为未来的技术迁移做好准备。
- 采用安全且符合行业最佳实践的密码机制。
- 建立安全的密钥管理体系，并实施适当的访问控制和审计。
- 定期评估密码技术的发展和应用情况，识别新的风险，并相应调整所用算法。
- 在应用的整个生命周期内识别和管理密码技术的应用场景，确保所有相关资产均已登记并得到妥善保护。

除了一般原则和最佳实践，附录 C “密码学标准” 还提供了更深入的技术说明，并列出了本章认可使用的算法和工作模式。

秘密管理、通信安全等具体场景所涉及的密码技术要求，分别列在本标准的其他章节。

V11.1 密码资产清单和文档

应用需要建立安全可靠的密码技术架构，根据数据分类采取相应的保护措施。所有内容一律加密会浪费资源；完全不加密又可能违反法律义务。二者之间的取舍应在架构设计、高层设计、设计冲刺或架构探索阶段完成。等到开发后期再临时拼凑或补救密码技术方案，成本通常远高于从一开始就做好设计。

确保定期识别、清点和评估所有密码资产至关重要。具体方法请参见附录。

密码技术体系还需要能够应对未来量子计算发展带来的安全风险。后量子密码学（PQC）指为抵御量子计算机攻击而设计的密码算法；量子计算机预计能够破解 RSA 和椭圆曲线密码学（ECC）等广泛使用的算法。

关于经过审查的 PQC 原语和标准的当前指导，请参见附录。

#	描述	级别
11.1.1	验证应用制定了书面的密钥管理策略，并按照 NIST SP 800-57 等标准管理密钥的整个生命周期。策略应防止密钥被过度共享；例如，共享秘密最多只供两个实体使用，私钥只能由一个实体持有。	2
11.1.2	验证应用建立并持续更新密码资产清单，对应用使用的所有密钥、算法和证书进行登记。对于每个密钥，清单还必须说明其可以和不可以系统的哪些位置使用，以及可以和不可以保护哪些类型的数据。	2
11.1.3	验证应用使用相应的工具或机制，识别系统中的所有密码运算，包括加密、哈希和数字签名。	3
11.1.4	验证应用持续维护密码资产清单，并在其中记录迁移到新密码技术标准（例如后量子密码学）的计划和路径，以应对未来威胁。	3

V11.2 密码技术的安全实现

本节规定如何选择、实现和持续管理应用的核心密码算法，目标是确保只部署安全可靠、得到业界认可的密码学原语，并符合现行标准（例如 NIST、ISO/IEC）和最佳实践。组织必须确保每个密码组件的选择都以经过同行评审的证据和实际安全测试为依据。

#	描述	级别
11.2.1	验证所有密码运算都采用经过业界检验的实现，包括软件库和硬件加速实现。	2
11.2.2	验证应用在设计上具备密码敏捷性，可以随时重新配置、升级或替换随机数生成算法、认证加密算法、MAC 或哈希算法、密钥长度、迭代轮数、密码算法和工作模式，以应对密码机制被攻破的风险。同样，应用还必须能够更换密钥和密码，并重新加密数据。这样，当获认可的后量子密码方案或标准的高保证实现广泛可用时，应用就能平稳升级到后量子密码学（PQC）。	2
11.2.3	验证所有密码学原语在所用算法、密钥长度和配置下，至少达到 128 位安全强度。例如，256 位 ECC 密钥约等于 128 位安全强度；RSA 则需要 3072 位密钥才能达到同等水平。	2
11.2.4	验证所有密码运算均采用常数时间实现，比较、计算和返回过程中不会因“短路”而提前结束，从而避免通过执行时间泄露信息。	3
11.2.5	验证所有密码模块在发生故障时都能保持安全，且错误处理方式不会导致填充预言机攻击等漏洞。	3

V11.3 加密算法

采用基于 AES 或 ChaCha20 的认证加密算法，是现代密码技术实践的基础。

#	描述	级别
11.3.1	验证应用没有使用 ECB 等不安全的分组密码工作模式，也没有使用 PKCS#1 v1.5 等强度不足的填充方案。	1
11.3.2	验证应用只使用本标准认可的加密算法和工作模式，例如 AES-GCM。	1
11.3.3	验证加密数据同时受到完整性保护，攻击者无法在未获授权的情况下修改数据。应优先采用本标准认可的认证加密算法；也可以把本标准认可的加密算法与本标准认可的 MAC 算法组合使用。	2
11.3.4	验证同一个 nonce、初始化向量或其他一次性数值不会在不同的“加密密钥与数据”组合中重复使用。生成这些数值的方法必须符合所用算法的要求。	3
11.3.5	验证加密算法与 MAC 算法组合使用时，始终采用“先加密、后计算 MAC”（Encrypt-then-MAC）的顺序。	3

V11.4 哈希和基于哈希的函数

安全哈希函数用于数字签名、HMAC、密钥派生函数（KDF）、随机位生成和密码存储等多种密码协议。整个密码技术体系的安全性取决于底层哈希函数的强度。本节说明密码运算使用安全哈希函数时需要满足的要求。

关于密码存储和附录中的相关内容，还可以参考 OWASP Password Storage Cheat Sheet。

#	描述	级别
11.4.1	验证数字签名、HMAC、KDF、随机位生成等需要密码学安全性的用途只采用本标准认可的哈希函数。MD5 等被禁止的哈希函数不得用于任何需要密码学安全性的场景。	1
11.4.2	验证密码使用本标准认可且计算成本足够高的密钥派生函数（也称“密码哈希函数”）进行存储，并按照最新指导配置参数。参数需要兼顾安全性和性能，使暴力破解在目标安全等级下足够困难。	2
11.4.3	验证数字签名、数据认证或完整性保护所使用的哈希函数具有所需的抗碰撞能力和足够的输出长度。需要抗碰撞时，输出长度至少为 256 位；只需抵抗第二原像攻击时，输出长度至少为 128 位。	2
11.4.4	验证应用从密码派生秘密密钥时，使用本标准认可的密钥派生函数，并设置密钥拉伸参数。这些参数必须兼顾安全性和性能，防止暴力破解攻击危及派生出的密码学密钥。	2

V11.5 随机值

密码学安全伪随机数生成器（CSPRNG）很难自行正确实现。系统中的高质量熵源可能因过度使用而耗尽；随机性不足的来源又会生成可预测的密钥和秘密。

#	描述	级别
11.5.1	验证所有需要防止猜测的随机数和随机字符串都由密码学安全伪随机数生成器 (CSPRNG) 生成，并且至少包含 128 位熵。UUID 不满足此要求。	2
11.5.2	验证随机数生成机制即使在请求量很高时也能安全、可靠地工作。	3

V11.6 公钥密码学

公钥密码技术适用于多个参与方之间无法共享或不宜共享秘密密钥的场景。

为确保所用密码机制能够抵御现代威胁，需要采用本标准认可的密钥交换机制，例如 Diffie-Hellman 和椭圆曲线 Diffie-Hellman (ECDH)。“安全通信”章节已经规定了 TLS 要求，因此本节适用于在 TLS 之外使用公钥密码学的场景。

#	描述	级别
11.6.1	验证生成密钥、为密钥生成过程提供随机种子，以及生成和验证数字签名时，只采用本标准认可的密码算法和工作模式。密钥生成算法不得产生容易受到已知攻击的弱密钥，例如容易被 Fermat 分解法破解的 RSA 密钥。	2
11.6.2	验证 Diffie-Hellman 等密钥交换使用本标准认可的密码算法和安全参数，防止攻击者破坏密钥建立过程，进而实施中间人攻击或破坏密码机制的安全性。	3

V11.7 使用中数据的密码保护

保护正在处理中的数据至关重要。建议采用全内存加密、传输中数据加密等技术，并确保数据在使用后尽快加密。

#	描述	级别
11.7.1	验证应用采用全内存加密，在敏感数据处理期间防止未经授权的用户或进程访问这些数据。	3
11.7.2	验证应用遵循数据最小化原则，把处理过程中暴露的数据降到最低，并在使用完毕后立即或在可行的情况下尽快加密。	3

参考资料

更多信息，另见：

- OWASP Web Security Testing Guide: Testing for Weak Cryptography
- OWASP Cryptographic Storage Cheat Sheet

- FIPS 140-3
- NIST SP 800-57

V12 安全通信

控制目标

本章规定如何保护传输中的数据，既包括终端用户客户端与后端服务之间的通信，也包括应用内部各服务之间的通信。

本章主要强调以下安全原则和最佳实践：

- 确保对外通信已加密，理想情况下内部通信也应加密。
- 按照现行安全建议配置通信加密机制，包括选用当前推荐的算法和密码套件。
- 通过使用签名证书，确保通信不被未经授权方拦截。

除上述安全原则和最佳实践外，附录 C “密码学标准” 还提供了有关密码机制安全强度的详细技术说明。

V12.1 通用 TLS 安全指导

本节给出保护 TLS 通信的基本要求。应用应持续使用最新工具检查 TLS 配置。

通配符 TLS 证书本身并非不安全。但如果同一张证书部署在所有自有环境中（例如生产、预发布、开发和测试环境），该证书遭到攻破可能危及使用它的应用的安全。因此，应尽可能妥善保护和管理证书，并在不同环境中使用独立的 TLS 证书。

#	描述	级别
12.1.1	验证应用只启用当前仍受推荐的 TLS 版本，例如 TLS 1.2 和 TLS 1.3，并优先使用其中最新的版本。	1
12.1.2	验证应用只启用当前推荐的密码套件，并优先选择其中安全强度最高的套件。L3 应用只能支持具备前向保密性的密码套件。	2
12.1.3	验证应用在根据 mTLS 客户端证书的身份进行认证或授权之前，先确认该证书可信。	2
12.1.4	验证应用启用并配置了适当的证书吊销机制，例如在线证书状态协议（OCSP）的响应附带机制（OCSP Stapling）。	3
12.1.5	验证应用的 TLS 配置启用了加密 ClientHello（ECH），以防止服务器名称指示（SNI）等敏感元数据在 TLS 握手过程中暴露。	3

V12.2 与面向外部的服务进行 HTTPS 通信

应用所有对外提供的 HTTP 服务都必须使用加密连接，并采用公开受信任的证书。

#	描述	级别
12.2.1	验证客户端与所有对外 HTTP 服务之间的连接都使用 TLS，并且不会降级为不安全或未加密的通信。	1
12.2.2	验证面向外部的服务使用公开受信任的 TLS 证书。	1

V12.3 通用服务间通信安全

服务器对内和对外的通信并不只有 HTTP。应用与其他系统之间的所有入站和出站连接也必须受到保护，并应尽可能使用 TLS。

#	描述	级别
12.3.1	验证所有进出应用的连接都使用 TLS 等加密协议，包括与监控系统、管理工具、远程访问和 SSH 服务、中间件、数据库、大型机、合作伙伴系统及外部 API 的连接。服务不得降级为不安全或未加密的协议。	2
12.3.2	验证 TLS 客户端在与服务器交换数据之前，会检查服务器提供的证书是否可信且有效。	2
12.3.3	验证应用内部基于 HTTP 的服务之间全部使用 TLS 或其他合适的传输加密机制，而且不会降级为不安全或未加密的通信。	2
12.3.4	验证内部服务之间的 TLS 连接使用可信证书。如果证书由内部 CA 签发，发起连接的服务必须只信任明确指定的内部 CA；如果使用自签名证书，则必须只信任明确指定的自签名证书。	2
12.3.5	验证系统内部相互通信的服务采用强认证方式，以确认每个端点的身份。这里的强认证必须基于公钥基础设施，并能抵抗重放攻击，例如 TLS 客户端认证。微服务架构可以使用服务网格来简化证书管理并加强安全控制。	3

参考资料

更多信息，另见：

- OWASP - Transport Layer Security Cheat Sheet
- Mozilla' s Server Side TLS configuration guide
- Mozilla' s tool to generate known good TLS configurations.
- O-Saft - OWASP Project to validate TLS configuration

V13 配置

控制目标

应用采用默认配置并在互联网环境中使用时，必须是安全的。

本章说明在开发、构建和部署阶段应如何配置应用，才能达到这一目标。

覆盖主题包括防止数据泄露、安全管理组件之间的通信，以及保护秘密。

V13.1 配置相关文档

本节规定应用文档需要记录哪些与配置和运行有关的信息，包括应用如何与内部和外部服务通信、如何避免服务不可用导致应用失去可用性，以及如何记录应用使用的秘密。

#	描述	级别
13.1.1	验证应用文档完整记录了所有通信需求，包括应用依赖的外部服务，以及最终用户可以提供外部目标地址、再由应用连接该地址的场景。	2
13.1.2	验证应用文档针对所使用的每项服务，规定了最大并发连接数（例如连接池上限），并说明达到上限时应用如何处理，包括采用哪些回退或恢复机制来防止拒绝服务。	3
13.1.3	验证应用文档为应用使用的每个外部系统或服务（例如数据库、文件句柄、线程和 HTTP 连接）规定了资源管理策略。策略应包括资源释放流程、超时设置和故障处理；实现了重试逻辑时，还应规定重试次数上限、间隔和退避算法。对于同步 HTTP 请求与响应操作，策略应要求设置较短的超时时间，并禁用重试或严格限制重试，以防止延迟级联和资源耗尽。	3
13.1.4	验证应用文档列出了对应用安全至关重要的秘密，并根据组织的威胁模型和业务要求规定了轮换计划。	3

V13.2 后端通信配置

应用会与 API、数据库等多个服务或组件交互。这些对象可能属于应用内部，但不受应用常规访问控制机制管理；也可能完全由外部提供。无论属于哪种情况，都必须对应用与这些组件之间的交互进行安全配置；必要时，还必须防止相关连接配置被未授权查看或修改。

注意：“安全通信”章节提供了传输中加密指导。

#	描述	级别
13.2.1	验证 API、中间件和数据层等后端组件，在无法使用应用常规用户会话机制时，会对彼此之间的通信进行身份认证。认证必须使用各自独立的服务账号、短期令牌或基于证书的认证，不得使用密码、API 密钥或具有特权的共享账号等长期不变的凭据。	2
13.2.2	验证本地服务、操作系统服务、API、中间件和数据层等后端组件相互通信时，使用的账号只拥有完成任务所需的最低权限。	2
13.2.3	验证服务认证需要使用凭据时，调用方没有使用 root/root、admin/admin 等默认凭据。	2
13.2.4	验证应用通过允许列表限制可以通信的外部资源和系统，包括出站请求、数据加载和文件访问的目标。允许列表可以在应用、Web 服务器、防火墙或多个层面共同实施。	2
13.2.5	验证 Web 服务器或应用服务器配置了目标允许列表，明确限制服务器可以向哪些地址发送请求，以及可以从哪些位置加载数据或文件。	2
13.2.6	验证应用连接独立服务时，遵循应用文档为每个连接规定的配置，包括最大并发连接数、达到上限后的处理方式、连接超时和重试策略。	3

V13.3 秘密管理

秘密管理是保护应用所用数据的一项基本配置任务。密码学的具体要求位于“密码学”章节，而本节聚焦于秘密的管理和处理。

#	描述	级别
13.3.1	验证应用使用密钥库等专门的秘密管理系统，安全地生成、存储和销毁应用后端使用的秘密，并控制访问权限。此类秘密可以包括密码、密钥材料、连接数据库和第三方系统所需的凭据、基于时间的令牌所需的密钥和种子，以及 API 密钥等内部秘密。秘密不得写入源代码或构建产物。L3 应用必须使用硬件安全模块（HSM）等由硬件提供安全保护的方案。	2
13.3.2	验证对秘密的访问遵循最小权限原则。	2
13.3.3	验证所有密码运算都通过隔离的安全模块执行，例如密钥库或硬件安全模块（HSM）。密钥材料必须始终由该模块安全管理和保护，不得暴露到模块之外。	3
13.3.4	验证秘密已按照应用文档配置到期失效和轮换机制。	3

V13.4 非预期信息泄露

生产环境应经过配置加固，避免泄露不必要的信息。这类问题单独出现时往往不构成重大风险，却经常被攻击者与其他漏洞组合利用。默认关闭这些信息泄露渠道，可以提高攻击难度。

例如，隐藏服务端组件版本并不能消除修补所有组件的需要；禁用目录列表也不能消除使用授权控制或避免文件放入公共文件夹的需要，但它确实提高了门槛。

#	描述	级别
13.4.1	验证部署产物不包含 .git、.svn 等版本控制元数据。如果确实存在这些目录，则无论外部用户还是应用自身都不能访问。	1
13.4.2	验证生产环境中的所有组件都已关闭调试模式，避免暴露调试功能或泄露内部信息。	2
13.4.3	验证 Web 服务器默认不向客户端显示目录列表；只有明确预期提供目录列表时才可以启用。	2
13.4.4	验证生产环境不支持 HTTP TRACE 方法，以避免潜在信息泄露。	2
13.4.5	验证文档（例如内部 API 文档）和监控端点不会暴露，除非明确预期公开这些内容。	2
13.4.6	验证应用不会暴露后端组件的详细版本信息。	3
13.4.7	验证 Web 层（例如 Web 服务器）只向客户端提供扩展名已明确允许的文件，防止内部信息、配置文件或源代码被意外公开。	3

参考资料

更多信息，另见：

- OWASP Web Security Testing Guide: Configuration and Deployment Management Testing

V14 数据保护

控制目标

应用无法掌控所有使用方式和用户行为，因此应实施控制，限制对客户端设备上敏感数据的未授权访问。

本章说明如何识别需要保护的数据、如何确定保护方式，以及应采用哪些机制、避免哪些常见问题。

数据保护还要考虑批量提取、批量修改和过度使用数据。不同系统对“异常行为”的定义差异很大，必须结合威胁模型和业务风险判断。在 ASVS 中，这类行为的检测要求见“安全日志和错误处理”章节，使用限制见“验证和业务逻辑”章节。

V14.1 数据保护文档

保护数据的前提是先识别哪些数据属于敏感数据，并划分敏感等级。不同等级的数据需要采用不同强度的保护措施。

有多种隐私法规和法律会影响应用必须如何存储、使用和传输敏感个人信息。本节不再试图重复这些数据保护或隐私法律，而是聚焦于保护敏感数据的关键技术考虑。请查阅当地法律法规，并按需咨询合格的隐私专家或律师。

#	描述	级别
14.1.1	验证应用已经识别其创建和处理的所有敏感数据，并为每类数据指定保护等级。仅经过编码、很容易恢复原文的数据也属于识别范围，例如 Base64 字符串或 JWT 中的明文载荷。划分等级时，必须考虑应用需要遵守的数据保护和隐私法律、法规及标准。	2
14.1.2	验证每个敏感数据保护等级都有书面的保护要求。要求必须涵盖但不限于加密、完整性校验、保留期限、日志记录方式、日志中敏感数据的访问控制、数据库级加密、隐私保护和隐私增强技术，以及其他保密措施。	2

V14.2 通用数据保护

本节给出数据保护的具体要求。多数要求针对意外泄露等特定问题，同时也要求按照每项数据的保护等级落实相应控制。

#	描述	级别
14.2.1	验证向服务器发送敏感数据时，只把数据放在 HTTP 消息正文或头字段中。URL 和查询字符串不得包含 API 密钥、会话令牌等敏感信息。	1
14.2.2	验证应用防止负载均衡器、应用缓存等服务端组件缓存敏感数据，或确保这些数据在使用后得到安全清除。	2
14.2.3	验证应用不会把已经认定为敏感的数据发送给用户跟踪器等不可信方，防止数据脱离应用控制后被不必要地收集。	2
14.2.4	验证应用按照相应保护等级的文档要求，对敏感数据实施加密、完整性校验、保留期限、日志记录、日志访问控制、隐私保护和隐私增强等措施。	2
14.2.5	验证缓存系统仅缓存同时满足以下条件的响应：响应的 Content-Type 与该资源应有的类型一致，并且不包含动态生成的敏感内容。如果请求的文件不存在，Web 服务器应返回 404 或 302 响应，而不是返回另一个实际存在的文件，以防止 Web 缓存欺骗（Web Cache Deception）攻击。	3
14.2.6	验证应用只返回实现当前功能所必需的最少敏感数据。例如，只返回信用卡号的部分数字，而不是完整卡号。如果需要完整数据，应在用户界面中将其遮蔽，除非用户明确选择查看。	3
14.2.7	验证敏感信息受数据保留分类规则约束，确保过时或不再需要的数据会被自动删除、按既定计划删除，或在具体情况需要时删除。	3
14.2.8	验证应用会从用户提交文件的元数据中删除敏感信息；只有用户明确同意保留时才可以例外。	3

V14.3 客户端数据保护

本节包含防止数据在应用客户端或用户代理侧以特定方式泄露的要求。

#	描述	级别
14.3.1	验证客户端关闭或会话终止后，客户端存储（例如浏览器 DOM）中的已认证数据会被清除。Clear-Site-Data 响应头可以帮助完成清理；但即使会话终止时无法连接服务器，客户端也应能够自行清除数据。	1
14.3.2	验证应用设置了足以防止缓存的 HTTP 响应头字段（即 Cache-Control: no-store），使敏感数据不会被浏览器缓存。	2
14.3.3	验证 localStorage、sessionStorage、IndexedDB、Cookie 等浏览器存储中不保存敏感数据；会话令牌是唯一例外。	2

参考资料

更多信息，另见：

- Consider using the Security Headers website to check security and anti-caching header fields
- Documentation about anti-caching headers by Mozilla
- OWASP Secure Headers project
- OWASP Privacy Risks Project
- OWASP User Privacy Protection Cheat Sheet
- Australian Privacy Principle 11 - Security of personal information
- European Union General Data Protection Regulation (GDPR) overview
- European Union Data Protection Supervisor - Internet Privacy Engineering Network
- Information on the “Clear-Site-Data” header
- White paper on Web Cache Deception

V15 安全编码和架构

控制目标

许多 ASVS 要求要么与认证、授权等特定安全领域相关，要么与日志、文件处理等特定类型的应用功能相关。

本章给出设计和开发应用时普遍适用的安全要求，既涉及清晰的架构和代码质量，也涉及保障应用安全所必需的架构设计与编码实践。

V15.1 安全编码和架构文档

要建立安全且具备防御能力的架构，许多要求都依赖清晰的文档，以记录特定安全控制和应用所用组件的实现决策。

本节概述文档要求，包括识别被视为包含“危险功能”或属于“风险组件”的组件。

“危险功能”可能存在于内部组件或第三方组件中，包括反序列化不可信数据、解析原始文件或二进制数据、动态执行代码，以及直接操作内存。这类功能一旦出现漏洞，往往会导致应用被攻破，甚至危及底层基础设施。

“风险组件”是指开发流程或功能缺少安全控制，或者安全控制实现较差的第三方库（即非内部开发库）。例如维护不佳、不再受支持、已到生命周期末期或曾出现重大漏洞的组件。

本节还强调为修复第三方组件漏洞定义合适时限的重要性。

#	描述	级别
15.1.1	验证应用文档根据风险规定了修复第三方组件已知漏洞的时限，以及常规更新依赖库的时限，从而尽量降低组件风险。	1
15.1.2	验证应用维护了所有在用第三方库的清单，例如软件物料清单（SBOM），并确认每个组件都来自预先指定、可信且持续维护的仓库。	2
15.1.3	验证应用文档列出了执行时间较长，或会大量占用计算、内存、连接等系统资源的功能，并说明如何防止这些功能被过度使用而导致应用不可用，以及如何避免响应生成时间超过调用方的超时限制。可采用异步处理、任务队列，并限制每个用户及整个应用同时执行的任务数量等措施。	2
15.1.4	验证应用文档标出被视为“风险组件”的第三方库。	3
15.1.5	验证应用文档标出应用中正在使用“危险功能”的部分。	3

V15.2 安全架构和依赖

本节说明如何通过依赖管理处理高风险、过时或不安全的依赖和组件。

本节还要求采用沙箱、封装、容器化和网络隔离等架构措施，限制“危险功能”或“风险组件”遭到利用后的影响，并防止大量占用系统资源的功能被滥用而导致应用不可用。

#	描述	级别
15.2.1	验证应用中的所有组件都仍处于文档规定的更新和漏洞修复期限内。	1
15.2.2	验证应用按照书面的安全决策和策略实施了防护，避免执行时间较长或大量占用系统资源的功能影响应用可用性。	2
15.2.3	验证生产环境只包含应用运行所必需的功能，不暴露测试代码、示例片段和开发功能等额外功能。	2
15.2.4	验证第三方组件及其所有传递依赖均来自预期的仓库，无论仓库由内部持有还是来自外部，并且不存在依赖混淆攻击的风险。	3
15.2.5	验证应用对文档中标记为包含“危险功能”或使用“风险组件”第三方库的部分实施额外保护。保护措施可以包括沙箱、封装、容器化或网络级隔离等技术，以延缓并阻止已攻破应用某一部分的攻击者横向移动到应用其他位置。	3

V15.3 防御式编码

本节讨论由编程语言中的不安全写法引发的漏洞，例如类型转换混淆和原型污染。有些漏洞只出现在特定语言中，也可能需要采用特定语言的修复方式；另一些则取决于语言或框架如何处理 HTTP 参数等功能。本节还考虑未通过密码学手段验证应用更新所带来的风险。

本节还讨论应用通过外部 API 接收或返回数据对象时可能产生的风险。接收数据时，应用必须防止用户通过输入修改本不允许其修改的字段（即批量赋值）；返回数据时，API 必须只返回必要的字段。如果用户能否访问某个字段取决于其权限，还必须遵循“授权”章节中有关字段级访问控制的要求。

#	描述	级别
15.3.1	验证应用只返回数据对象中所需的字段子集。例如，不应返回整个数据对象，因为其中某些字段不应供用户访问。	1
15.3.2	验证应用后端调用外部 URL 时，配置为不跟随重定向，除非跟随重定向是预期功能。	2
15.3.3	验证应用针对每个控制器和操作限制允许的字段，以防范批量赋值攻击。例如，用户不得通过请求插入或更新该功能原本不允许处理的字段。	2
15.3.4	验证所有代理和中间件都通过最终用户无法篡改的可信字段传递用户原始 IP 地址，应用和 Web 服务器则使用这个可信值进行日志记录、速率限制等安全决策。同时应认识到，动态 IP、VPN 和企业防火墙等因素仍会降低原始 IP 地址的可靠性。	2
15.3.5	验证应用明确确保变量类型正确，并执行严格的相等和比较运算，以避免应用代码对变量类型作出假设而导致类型转换混淆或类型混淆漏洞。	2
15.3.6	验证 JavaScript 代码采用能够防范原型污染的写法，例如使用 Set() 或 Map()，而不是使用对象字面量。	2
15.3.7	验证应用具备防御 HTTP 参数污染攻击的措施，尤其是在应用框架不区分请求参数来源（查询字符串、正文参数、Cookie 或头字段）时。	2

V15.4 安全并发

并发问题，例如竞态条件、检查时到使用时（TOCTOU）漏洞、死锁、活锁、线程饥饿和不当同步，可能导致不可预测行为和安全风险。本节包含帮助缓解这些风险的多种技术和策略。

#	描述	级别
15.4.1	验证多线程代码通过线程安全类型以及锁、信号量等同步机制来访问共享对象，例如缓存、文件或多线程共用的内存对象，以避免竞态条件和数据损坏。	3
15.4.2	验证对资源状态（例如是否存在或权限）的检查及依赖检查结果的操作，作为单个原子操作执行，以防范检查时到使用时（TOCTOU）竞态条件。例如，打开文件前检查文件是否存在，或授予用户访问权限前验证其访问权限。	3

#	描述	级别
15.4.3	验证应用采用一致的加锁规则，避免线程因相互等待或无限重试而卡死。加锁逻辑保留在负责管理相应资源的代码中，防止外部类或其他代码无意或恶意地修改锁。	3
15.4.4	验证资源分配策略保证各线程能够公平获得资源，防止线程饥饿。例如，使用线程池并确保低优先级线程也能在合理时间内继续执行。	3

参考资料

更多信息，另见：

- OWASP Prototype Pollution Prevention Cheat Sheet
- OWASP Mass Assignment Prevention Cheat Sheet
- OWASP CycloneDX Bill of Materials Specification
- OWASP Web Security Testing Guide: Testing for HTTP Parameter Pollution

V16 安全日志和错误处理

控制目标

安全日志不同于错误日志和性能日志，它专门记录认证、访问控制等安全决策，以及绕过输入验证、业务逻辑检查等安全控制的尝试。安全日志应提供噪声少、价值高的结构化数据，供 SIEM 等工具分析，以支持安全检测、事件响应和调查取证。

除非法律要求，日志不应包含敏感个人数据；任何记录到日志中的数据都必须作为高价值资产加以保护。日志记录不得损害隐私或系统安全。应用还必须能够安全地处理故障，避免不必要的信息披露或服务中断。

详细实现指导请参考参考资料章节中的 OWASP Cheat Sheets。

V16.1 安全日志文档

本节要求完整记录应用技术栈各层的日志机制，为安全监控、事件响应和合规工作提供基础。

#	描述	级别
16.1.1	验证存在一份清单，记录应用技术栈各层的日志记录情况，包括记录哪些事件、日志格式、存储位置、用途、访问控制方式及保留期限。	2

V16.2 通用日志记录

本节提供要求，确保安全日志结构一致并包含预期元数据。目标是让日志在分布式系统和工具中可被机器读取和分析。

安全事件往往涉及敏感数据。如果未经审慎考虑便记录这些数据，日志本身就会被归类为敏感资产，因而需要满足加密要求和更严格的保留策略，并且可能在审计期间被披露。

因此，只记录必要内容，并像对待其他敏感资产一样谨慎对待日志数据至关重要。

以下要求建立了日志元数据、同步、格式和控制方面的基础要求。

#	描述	级别
16.2.1	验证每条日志都包含必要的元数据（例如何时、何地、何人、何事），以便在事件发生时详细调查事件的时间线。	2
16.2.2	验证所有日志组件的时间源均已同步，且安全事件元数据中的时间戳使用 UTC 或明确包含时区偏移。推荐统一使用 UTC，以便在分布式系统中保持一致，并避免夏令时切换造成混淆。	2
16.2.3	验证应用只将日志写入日志清单中列明的文件，或发送给清单中列明的日志服务。	2
16.2.4	验证现有日志处理工具能够读取并关联各类日志；最好统一采用通用日志格式。	2
16.2.5	验证应用记录敏感数据时，按照相应的数据保护等级执行日志规则。例如，凭据和支付信息可能完全禁止记录；会话令牌等数据则可能只有在全部或部分经过哈希或遮蔽处理后才允许记录。	2

V16.3 安全事件

本节定义在应用内记录安全相关事件的要求。捕获这些事件对于检测可疑行为、支持调查和履行合规义务至关重要。

本节概述应记录的事件类型，但不试图提供详尽细节。每个应用都有独特风险因素和运行上下文。

注意，虽然 ASVS 将安全事件日志记录纳入范围，但告警和关联（例如 SIEM 规则或监控基础设施）被视为范围外内容，由运维和监控系统处理。

#	描述	级别
16.3.1	验证应用记录所有认证操作，包括成功和失败的尝试。还应收集额外的元数据，例如认证类型或所使用的认证因素。	2
16.3.2	验证应用记录所有失败的授权尝试。L3 应用还必须记录全部授权决策，包括何时访问了敏感数据，但日志中不得包含敏感数据本身。	2

#	描述	级别
16.3.3	验证应用记录文档中定义的安全事件，以及试图绕过输入验证、业务逻辑、反自动化等安全控制的行为。	2
16.3.4	验证应用记录意外错误和安全控制失败，例如后端 TLS 失败。	2

V16.4 日志保护

日志是有价值的取证材料，必须受到保护。如果日志可以被轻易修改或删除，它们就会失去完整性，并且在事件调查或法律程序中变得不可靠。日志可能暴露应用内部行为或敏感元数据，因此也是攻击者感兴趣的目标。

本节定义要求，确保日志免受未经授权访问、篡改和披露，并确保日志被安全传输和存储在安全、隔离的系统中。

#	描述	级别
16.4.1	验证所有日志组件都对数据进行适当编码，以防止日志注入。	2
16.4.2	验证日志受到保护，免遭未经授权访问，且不可被修改。	2
16.4.3	验证日志通过安全通道传输到逻辑上独立的系统，用于分析、检测、告警和事件升级。目的是确保即使应用被攻破，日志也不会遭到破坏。	2

V16.5 错误处理

本节要求应用在发生故障时妥善、安全地处理，不向外泄露敏感的内部细节。

#	描述	级别
16.5.1	验证发生意外错误或涉及安全的错误时，只向使用方返回通用错误消息，不暴露栈跟踪、查询语句、秘密密钥、令牌等系统内部敏感信息。	2
16.5.2	验证外部资源访问失败时，应用仍能安全运行，例如采用断路器或优雅降级等模式。	2
16.5.3	验证应用在发生故障时（包括发生异常时）能够妥善、安全地处理，防止出现失效开放 (fail-open) 的情况，例如在验证逻辑报错后仍处理事务。	2
16.5.4	验证应用设置了能够捕获所有未处理异常的兜底错误处理器。这是为了避免丢失必须写入日志文件的错误详情，同时确保错误不会导致整个应用进程终止，造成可用性丧失。	3

注意：Swift、Go 以及许多采用常见设计方式的函数式语言不支持异常或全局兜底处理器。使用这类语言时，架构师和开发人员应遵循相应语言、框架和设计模式的惯用做法，确保应用仍能安全处理错误、意外情况和安全事件。

参考资料

更多信息，另见：

- OWASP Web Security Testing Guide: Testing for Error Handling
- OWASP Authentication Cheat Sheet section about error messages
- OWASP Logging Cheat Sheet
- OWASP Application Logging Vocabulary Cheat Sheet

V17 WebRTC

控制目标

Web实时通信（WebRTC）使应用能够实时交换语音、视频和数据。随着 WebRTC 日益普及，保护相关基础设施也越来越重要。本章面向开发、托管或集成 WebRTC 系统的相关方，给出相应的安全要求。

WebRTC 市场大致可分为三个部分：

1. 产品开发者：开发并提供专有或开源 WebRTC 产品与解决方案的厂商，重点是为其他组织提供稳健、安全的 WebRTC 技术。
2. 通信平台即服务（CPaaS）提供方：通过 API、SDK 和必要的基础设施或平台提供 WebRTC 能力。它们既可能采用第一类厂商的产品，也可能自行开发 WebRTC 软件。
3. 服务提供方：采用产品开发者或 CPaaS 提供方的产品，或者自行开发 WebRTC 解决方案，为在线会议、医疗、在线教育等依赖实时通信的场景提供应用。

这里概述的安全要求主要面向以下产品开发者、CPaaS 和服务提供方：

- 使用开源解决方案构建其 WebRTC 应用。
- 将商业 WebRTC 产品作为其基础设施的一部分。
- 使用内部开发的 WebRTC 解决方案，或将多个组件集成为统一服务。

如果开发人员完全通过 CPaaS 厂商提供的 SDK 和 API 使用 WebRTC，则本章要求并不适用。此时，大多数底层安全问题通常由 CPaaS 提供方负责，ASVS 这类通用标准也未必能完整覆盖开发人员需要关注的问题。

V17.1 TURN 服务器

本节适用于自行运行 TURN（Traversal Using Relays around NAT）服务器的系统。TURN 服务器负责在受限网络中转发媒体流，但配置不当也会引入风险，因此需要正确过滤目标地址并防止资源耗尽。

#	描述	级别
17.1.1	验证 TURN 服务禁止访问保留作特殊用途的 IP 地址，例如内部网络地址、广播地址和回环地址。此要求同时适用于 IPv4 和 IPv6。	2
17.1.2	验证合法用户尝试在 TURN 服务器上打开大量端口时，不会耗尽 TURN 服务的资源。	3

V17.2 媒体

这些要求只适用于自行部署并运行 WebRTC 媒体服务器的系统，例如选择性转发单元（SFU）、多点控制单元（MCU）、录制服务器或网关服务器。媒体服务器处理并分发媒体流，其安全性直接关系到参与通信的各个客户端之间的通信安全。在 WebRTC 应用中，保护媒体流是重中之重，以防止窃听、篡改和拒绝服务攻击危及用户隐私和通信质量。

媒体服务器尤其需要防范洪泛攻击。可以采用速率限制、校验时间戳、使用同步时钟匹配实际时间间隔，以及管理缓冲区防止溢出并维持正确时序。某个媒体会话的数据包到达过快时，应丢弃多余的数据包。系统还应通过输入验证、安全处理整数溢出、防止缓冲区溢出和稳健的错误处理来抵御畸形数据包。

完全依赖 Web 浏览器之间点对点媒体通信，且没有中间媒体服务器参与的系统，不适用这些特定媒体相关安全要求。

本节提到在 WebRTC 中使用数据报传输层安全（Datagram Transport Layer Security, DTLS）。关于记录密码学密钥管理策略的要求位于“密码学”章节。关于符合安全要求的密码方法，可参考 ASVS 密码学附录，也可参考 NIST SP 800-52 Rev. 2 或 BSI TR-02102-2（2025-01 版）等文档。

#	描述	级别
17.2.1	验证 DTLS 证书密钥按照书面的密码学密钥管理策略进行管理和保护。	2
17.2.2	验证媒体服务器配置为使用并支持获认可的 DTLS 密码套件，以及用于为安全实时传输协议建立密钥的 DTLS 扩展（DTLS-SRTP）的安全保护配置（protection profile）。	2
17.2.3	验证媒体服务器会校验安全实时传输协议（SRTP）的认证信息，防止实时传输协议（RTP）注入攻击导致拒绝服务，或将音频、视频插入媒体流。	2
17.2.4	验证媒体服务器在收到畸形安全实时传输协议（SRTP）数据包时，仍能正常处理传入的媒体流量。	2
17.2.5	验证媒体服务器在合法用户产生安全实时传输协议（SRTP）数据包洪泛时，仍能正常处理传入的媒体流量。	3
17.2.6	验证媒体服务器不受 DTLS ClientHello 竞态条件漏洞影响，验证方式为检查媒体服务器是否已知存在相关漏洞，或进行竞态条件测试。	3
17.2.7	验证与媒体服务器关联的音频或视频录制机制，在合法用户产生安全实时传输协议（SRTP）数据包洪泛时，仍能正常处理传入的媒体流量。	3

#	描述	级别
17.2.8	验证根据会话描述协议（SDP）的 fingerprint 属性校验 DTLS 证书，并在校验失败时终止媒体流，以确保媒体流的真实性。	3

V17.3 信令

本节适用于自行运行 WebRTC 信令服务器的系统。信令用于协调点对点通信，必须能够抵御可能干扰会话建立或控制的攻击。

信令服务器必须能够安全处理畸形输入，并在高负载下保持可用。

#	描述	级别
17.3.1	验证信令服务器在洪泛攻击期间仍能处理合法的传入信令消息。这应通过在信令层实施速率限制来实现。	2
17.3.2	验证信令服务器在收到可能导致拒绝服务的畸形信令消息时，仍能正常处理合法信令消息。可采用的措施包括输入验证、安全处理整数溢出、防止缓冲区溢出，以及其他稳健的错误处理技术。	2

参考资料

更多信息，另见：

- WebRTC DTLS ClientHello DoS 的最佳文档包括 Enable Security 面向安全专业人员的博客文章 以及相关的面向 WebRTC 开发人员的白皮书
- RFC 3550 - RTP: A Transport Protocol for Real-Time Applications
- RFC 3711 - The Secure Real-time Transport Protocol (SRTP)
- RFC 5764 - Datagram Transport Layer Security (DTLS) Extension to Establish Keys for the Secure Real-time Transport Protocol (SRTP)
- RFC 8825 - Overview: Real-Time Protocols for Browser-Based Applications
- RFC 8826 - Security Considerations for WebRTC
- RFC 8827 - WebRTC Security Architecture
- DTLS-SRTP Protection Profiles

附录 A：术语表

- **会话绝对最长生命周期**—NIST 也称其为“总体超时”。它指用户认证后，无论是否与应用交互，会话最多可以保持活动的时间。这是会话过期的一部分。
- **允许列表**—被允许的数据或操作列表，例如用于输入验证的一组允许字符。

- **防伪令牌**—一种在请求中传递一个或多个令牌，并由应用服务器验证这些令牌，以确保请求来自预期端点的机制。
- **应用安全**—侧重于分析构成开放系统互连参考模型（OSI 模型）应用层的组件，而非侧重于底层操作系统或所连接网络等方面。
- **应用安全验证**—根据 OWASP ASVS 对应用进行的技术评估。
- **应用安全验证报告**—由验证方针对特定应用生成的报告，记录总体结果和支撑分析。
- **认证**—验证应用用户所声称身份的过程。
- **自动化验证**—使用动态分析、静态分析或两者相结合的自动化工具，根据漏洞特征发现问题。
- **黑盒测试**—一种软件测试方法，在不查看应用内部结构或工作方式的情况下检查应用功能。
- **通用弱点枚举（CWE）**—由社区开发的常见软件安全弱点列表。它作为通用语言、软件安全工具的衡量标尺，以及弱点识别、缓解和预防工作的基线。
- **组件**—自包含的代码单元，带有相关磁盘和网络接口，并与其他组件通信。
- **凭据服务提供方（CSP）**—也称身份提供方（IdP）。一种用户数据来源，可被其他应用用作认证来源。
- **跨站脚本包含（XSSI）**—跨站脚本（XSS）攻击的一种变体，Web 应用从外部资源获取恶意代码，并把该代码作为自身内容的一部分包含进来。
- **跨站脚本（XSS）**—通常出现在 Web 应用中的安全漏洞，允许向内容中注入客户端脚本。
- **密码模块**—实现密码算法和/或生成密钥的硬件、软件和/或固件。
- **密码学安全伪随机数生成器（CSPRNG）**—具备适合密码学用途属性的伪随机数生成器，也称密码随机数生成器（CRNG）。
- **数据报传输层安全（DTLS）**—在网络连接上提供通信安全的密码协议。它基于 TLS 协议，但适配用于保护面向数据报的协议（通常基于 UDP）。DTLS 1.3 定义于 RFC 9147。
- **用于为安全实时传输协议建立密钥的 DTLS 扩展（DTLS-SRTP）**—使用 DTLS 握手为 SRTP 会话建立密钥材料的机制。定义于 RFC 5764。
- **设计验证**—对应用安全架构进行的技术评估。
- **动态应用安全测试（DAST）**—用于在应用运行状态下检测表明存在安全漏洞的情况的技术。
- **动态验证**—在应用执行期间，使用基于漏洞特征的自动化工具发现问题。
- **Fast IDentity Online（FIDO）**—一组认证标准，允许使用多种不同认证方法，包括生物识别、可信平台模块（TPM）、USB 安全令牌等。
- **硬件安全模块（HSM）**—以受保护方式存储密码学密钥和其他秘密的硬件组件。
- **Hibernate 查询语言（HQL）**—Hibernate ORM 库使用的一种查询语言，外观与 SQL 类似。
- **HTTP 严格传输安全（HSTS）**—一种策略，指示浏览器只在使用 TLS 且提供有效证书时连接返回该头的域名。它通过 Strict-Transport-Security 响应头字段启用。
- **超文本传输协议（HTTP）**—用于分布式、协作式、超媒体信息系统的协议，是万维网数据通信的基础。
- **基于 SSL/TLS 的 HTTP（HTTPS）**—通过使用传输层安全（TLS）加密来保护 HTTP 通信的方法。
- **身份提供方（IdP）**—在 NIST 参考资料中也称凭据服务提供方（CSP）。为其他应用提供认证来源的实体。
- **不活动超时**—在用户没有与应用交互的情况下，会话可以保持活动的时间长度。这是会话过期的一部分。
- **输入验证**—对不可信用户输入进行规范化和验证。
- **JSON Web Token（JWT）**—RFC 7519 定义的一种 JSON 数据对象标准，由说明如何验证对象的头部、包含一组声明的正文，以及包含数字签名的签名部分组成；该签名可用于验证正文内容。它是一种自包含令牌。
- **本地文件包含（LFI）**—一种利用应用中有漏洞的文件包含流程的攻击，会导致包含服务器上已经存在的本地文件。

- **恶意代码**–在应用所有者不知情的情况下，于开发期间被植入应用，并会绕过应用预期安全策略的代码。它不同于病毒、蠕虫等恶意软件。
- **恶意软件**–在运行时，在应用用户或管理员不知情的情况下引入应用的可执行代码。
- **消息认证码 (MAC)**–由 MAC 生成算法计算出的数据密码校验值，用于保证数据完整性和真实性。
- **多因素认证 (MFA)**–包含两个或更多单一因素的认证。
- **双向 TLS (mTLS)**–见 TLS 客户端认证。
- **对象关系映射 (ORM)**–一种系统，用于让应用通过与其兼容的对象模型引用和查询关系型或表格型数据库。
- **一次性密码 (OTP)**–为单次使用而唯一生成的密码。
- **Open Worldwide Application Security Project (OWASP)**–OWASP 是一个全球性、免费且开放的社区，专注于提升应用软件安全。其使命是让应用安全“可见”，使个人和组织能够对应应用安全风险作出知情决策。见：<https://www.owasp.org/>。
- **基于密码的密钥派生函数 2 (PBKDF2)**–一种特殊的单向算法，用于从输入文本（例如密码）和额外的随机盐值生成强密码学密钥。如果存储生成值而不是原始密码，可增加离线破解密码的难度。
- **公钥基础设施 (PKI)**–将公钥与相应实体身份绑定的体系。绑定通过证书颁发机构 (CA) 的注册和证书签发过程建立。
- **公共交换电话网络 (PSTN)**–包含固定电话和移动电话的传统电话网络。
- **实时传输协议 (RTP) 和实时传输控制协议 (RTCP)**–两个配合使用、用于传输多媒体流的协议。WebRTC 协议栈使用它们。定义于 RFC 3550。
- **引用令牌**–一种用作指针或标识符的令牌，指向存储在服务器上的状态或元数据，有时也称随机令牌或不透明令牌。自包含令牌会在令牌内嵌入相关数据，引用令牌则不携带这些信息，而要由服务器提供上下文。引用令牌本身可以是会话标识符，也可以包含会话标识符。
- **依赖方 (RP)**–通常指依靠独立认证提供方来认证用户的应用。应用根据认证提供方签发的令牌或一组签名断言，确认用户确实具有其所声明的身份。
- **远程文件包含 (RFI)**–一种利用应用中有漏洞的包含流程的攻击，会导致包含远程文件。
- **可缩放矢量图形 (SVG)**–一种基于 XML 的标记语言，用于描述二维矢量图形。
- **安全实时传输协议 (SRTP) 和安全实时传输控制协议 (SRTCP)**–RTP 和 RTCP 协议的一种配置 (profile)，支持消息加密、认证和完整性保护。定义于 RFC 3711。
- **安全架构**–对应应用设计的抽象，识别并描述安全控制在何处以及如何使用，也识别并描述用户数据和应用数据的位置与敏感性。
- **安全断言标记语言 (SAML)**–一种基于在身份提供方和依赖方之间传递签名断言（通常是 XML 对象）实现单点登录认证的开放标准。
- **安全配置**–影响安全控制如何使用的应用运行时配置。
- **安全控制**–执行安全检查（例如授权检查），或被调用后产生安全效果（例如生成审计记录）的功能或组件。
- **安全信息和事件管理 (SIEM)**–通过收集和分析组织 IT 基础设施内各来源的安全相关数据，实现威胁检测、合规和安全事件管理的系统。
- **自包含令牌**–一种封装一个或多个属性的令牌，这些属性不依赖服务端状态或其他外部存储。这类令牌确保其所含属性的真实性和完整性，支持系统间安全的“无状态”信息交换。自包含令牌通常使用数字签名或消息认证码 (MAC) 等密码技术来确保数据真实性、完整性，并在某些情况下确保保密性。常见示例包括 SAML 断言和 JWT。
- **服务端请求伪造 (SSRF)**–一种滥用服务器端功能读取或更新内部资源的攻击。攻击者提供或修改 URL，服务器上运行的代码会读取该 URL 或向其提交数据。

- **会话描述协议 (SDP)** –用于建立多媒体会话的消息格式（例如 WebRTC 中使用）。定义于 RFC 4566。
- **会话标识符或会话 ID** –用于标识后端所存储的有状态会话的键值。它会作为“引用令牌”或在“引用令牌”内部，在客户端和服务端之间传输。
- **会话令牌** –本标准中的统称，用于指代无状态会话机制（使用自包含令牌）或有状态会话机制（使用引用令牌）中使用的令牌或值。
- **NAT 会话穿越工具 (STUN)** –一种用于协助 NAT 穿越以建立点对点通信的协议。定义于 RFC 3489。
- **单因素认证器** –仅使用一种认证因素来确认用户身份的机制。该因素可以是用户知道的信息（记忆秘密、密码、口令短语或 PIN）、用户自身的特征（生物识别信息，如指纹或面部特征），或用户持有的物品（OTP 令牌、智能卡等密码设备）。
- **单点登录认证 (SSO)** –指用户登录一个应用后，无需重新认证就自动登录其他应用。例如，登录 Google 后，用户会自动登录 YouTube、Google Docs 和 Gmail 等其他 Google 服务。
- **软件物料清单 (SBOM)** –构建或组装软件应用所需所有组件、模块、库、框架和其他资源的结构化、完整列表。
- **软件成分分析 (SCA)** –一组用于分析应用组成、依赖、库和包的技术，以发现正在使用的特定组件版本中的安全漏洞。不要将其与源代码分析混淆，后者现在通常称为 SAST。
- **软件开发生命周期 (SDLC)** –软件从初始需求到部署和维护的逐步开发过程。
- **SQL 注入 (SQLi)** –一种用于攻击数据驱动应用的代码注入技术，将恶意 SQL 语句插入入口点。
- **有状态会话机制** –在有状态会话机制中，应用在后端保留会话状态，该状态通常对应一个会话令牌；该令牌使用密码学安全伪随机数生成器 (CSPRNG) 生成，并签发给最终用户。
- **无状态会话机制** –使用传递给客户端的自包含令牌来保存会话信息，接收并验证令牌的服务不一定另行存储这些信息。但在实际应用中，为了执行必要的安全控制，服务仍需访问部分会话信息，例如 JWT 撤销列表。
- **静态应用安全测试 (SAST)** –一组用于分析应用源代码、字节码和二进制文件的技术，以发现表明安全漏洞的编码和设计条件。SAST 解决方案在应用非运行状态下从“内部向外”分析应用。
- **威胁建模** –一种技术，通过逐步完善安全架构来识别威胁行为者、安全区域、安全控制以及重要技术和业务资产。
- **检查时到使用时 (TOCTOU)** –指应用在使用资源前检查资源状态，但在检查和使用之间资源状态可能发生变化。这会使检查结果失效，并因状态不一致导致应用执行无效操作。
- **基于时间的一次性密码 (TOTPs)** –一种生成 OTP 的方法，当前时间作为算法的一部分参与生成密码。
- **TLS 客户端认证**，也称 **双向 TLS (mTLS)** –在标准 TLS 连接中，客户端可以使用服务器提供的证书验证服务器身份。使用 TLS 客户端认证时，客户端也使用自己的私钥和证书，让服务器也能验证客户端身份。
- **传输层安全 (TLS)** –通过网络连接提供通信安全的密码协议。
- **Traversal Using Relays around NAT (TURN)** –STUN 协议的扩展，当无法建立直接点对点连接时，使用 TURN 服务器作为中继。定义于 RFC 8656。
- **可信执行环境 (TEE)** –一种隔离的处理环境，无论系统其余部分处于何种状态，应用都可以在其中安全执行。
- **可信平台模块 (TPM)** –一种 HSM，通常附加在主板等较大硬件组件上，并作为该系统的“信任根”。
- **可信服务层** –任何可信的控制执行点，例如微服务、无服务器 API、服务端、具备安全启动能力的客户端设备上的可信 API，以及合作伙伴或外部 API。所谓可信，是指不可信用户无法绕过该层，也无法跳过该层实施的控制。
- **统一资源标识符 (URI)** –标识资源的唯一字符串，例如网页、邮件地址或位置。
- **统一资源定位符 (URL)** –指定互联网资源位置的字符串。
- **通用唯一标识符 (UUID)** –软件中用作标识符的唯一参考编号。
- **验证方** –根据 OWASP ASVS 要求审查应用的个人或团队。

- **Web 实时通信 (WebRTC)**—一套协议栈及相关 Web API，用于在 Web 应用中传输多媒体流，通常用于电话会议场景。它基于 SRTP、SRTCP、DTLS、SDP 和 STUN/TURN。
- **基于 TLS 的 WebSocket (WSS)**—通过在 TLS 协议之上承载 WebSocket 来保护 WebSocket 通信的做法。
- **所见即所得 (WYSIWYG)**—一种富内容编辑器，显示内容渲染后的实际外观，而不是显示用于控制渲染的代码。
- **X.509 证书**—X.509 证书是一种数字证书，使用被广泛接受的国际 X.509 公钥基础设施 (PKI) 标准，验证公钥属于证书中包含的用户、计算机或服务身份。
- **XML 外部实体 (XXE)**—一种 XML 实体，可通过声明的系统标识符访问本地或远程内容。这可能导致多种注入攻击。

附录 B：参考资料

以下 OWASP 项目最可能对本标准的使用者和采用者有所帮助：

OWASP 核心项目

1. OWASP Top 10 Project: <https://owasp.org/www-project-top-ten/>
2. OWASP Web Security Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>
3. OWASP Proactive Controls: <https://owasp.org/www-project-proactive-controls/>
4. OWASP Software Assurance Maturity Model (SAMM): <https://owasp.org/www-project-samm/>
5. OWASP Secure Headers Project: <https://owasp.org/www-project-secure-headers/>

OWASP Cheat Sheet Series 项目

该项目包含多份与 ASVS 各主题相关的备忘单。

可在此处找到与 ASVS 的映射: <https://cheatsheetseries.owasp.org/IndexASVS.html>

移动安全相关项目

1. OWASP Mobile Security Project: <https://owasp.org/www-project-mobile-security/>
2. OWASP Mobile Top 10 Risks: <https://owasp.org/www-project-mobile-top-10/>
3. OWASP Mobile Security Testing Guide and Mobile Application Security Verification Standard: <https://owasp.org/www-project-mobile-security-testing-guide/>

OWASP 物联网相关项目

1. OWASP Internet of Things Project: <https://owasp.org/www-project-internet-of-things/>

OWASP Serverless 项目

1. OWASP Serverless Project: <https://owasp.org/www-project-serverless-top-10/>

其他

以下网站也可能对本标准的使用者和采用者有所帮助：

1. SecLists Github: <https://github.com/danielmiessler/SecLists>
2. MITRE 通用弱点枚举 (Common Weakness Enumeration, CWE): <https://cwe.mitre.org/>
3. PCI Security Standards Council: <https://www.pcisecuritystandards.org/>
4. PCI Data Security Standard (DSS) v3.2.1 Requirements and Security Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI_DSS_v3-2-1.pdf
5. PCI Software Security Framework - Secure Software Requirements and Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI-Secure-Software-Standard-v1_0.pdf
6. PCI Secure Software Lifecycle (Secure SLC) Requirements and Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI-Secure-SLC-Standard-v1_0.pdf
7. OWASP ASVS 4.0 Testing Guide: <https://github.com/BlazingWind/OWASP-ASVS-4.0-testing-guide>

附录 C：密码学标准

“密码学”章节不仅定义了最佳实践，还旨在帮助读者深入理解密码学原理，并推动采用更具韧性的现代安全方法。本附录针对各项要求提供详细的技术说明，作为“密码学”章节总体标准的补充。

本附录将密码机制分为以下使用级别：

- 可用 (A)：本标准认可，可在应用中使用。
- 遗留 (L)：不应在应用中使用，仅可用于兼容现有的遗留应用或代码。目前使用此类机制本身不被视为漏洞，但应尽快替换为更安全、能够适应未来需求的机制。
- 禁止 (D)：不得使用，因为这类机制目前已被攻破，或无法提供足够的安全性。

在具体应用中，可能需要基于多种原因调整此列表，包括：

- 密码学领域出现新的发展；
- 满足法规要求。

密码资产清单和文档

本节为 V11.1 密码资产清单和文档提供补充说明。

应定期发现、盘点和评估算法、密钥及证书等所有密码学资产。对于第 3 级，还应通过静态和动态扫描，识别应用中的密码学使用情况。SAST 和 DAST 工具可以提供帮助，但要实现更全面的覆盖，可能还需要使用专用工具。可免费使用的工具示例包括：

- CryptoMon：使用 eBPF、以 Python 编写的网络密码学监控工具
- Cryptobom Forge：根据 CodeQL 输出生成完整的 CBOM

密码学参数的等效安全强度

下表列出了不同密码学系统的相对安全强度（摘自 NIST SP 800-57 第 1 部分第 71 页）：

安全强度	对称密钥算法	有限域	整数分解	椭圆曲线
<= 80	2TDEA	L = 1024 N = 160	k = 1024	f = 160-223
112	3TDEA	L = 2048 N = 224	k = 2048	f = 224-255
128	AES-128	L = 3072 N = 256	k = 3072	f = 256-383
192	AES-192	L = 7680 N = 384	k = 7680	f = 384-511
256	AES-256	L = 15360 N = 512	k = 15360	f = 512+

应用示例：

- 有限域密码学：DSA、FFDH、MQV
- 整数分解密码学：RSA
- 椭圆曲线密码学：ECDSA、EdDSA、ECDH、MQV

注意：本节以不存在可用的量子计算机为前提。如果此类计算机成为现实，后三列的估算将不再有效。

随机值

本节为 V11.5 随机值提供补充说明。

名称	版本/参考资料	说明	状态
/dev/random	Linux 4.8+（2016 年 10 月），也存在于 iOS、Android 和其他基于 Linux 的 POSIX 操作系统中。基于 RFC7539	使用 ChaCha20 流。在正确配置的情况下，可通过 iOS 的 SecRandomCopyBytes 和 Android 的 SecureRandom 使用。	A
/dev/urandom	Linux 内核中用于提供随机数据的特殊文件	由硬件随机源提供高质量熵	A

名称	版本/参考资料	说明	状态
AES-CTR-DRBG	NIST SP800-90A	已用于常见实现，例如通过 BCRYPT_RNG_ALGORITHM 配置的 Windows CNG API BCryptGenRandom。	A
HMAC-DRBG	NIST SP800-90A		A
Hash-DRBG	NIST SP800-90A		A
getentropy()	OpenBSD，也可用于 Linux glibc 2.25+ 和 macOS 10.12+	通过简洁的 API 直接从内核熵源获取安全随机字节。该接口较为现代，可避免旧式 API 的常见问题。	A

HMAC-DRBG 或 Hash-DRBG 使用的底层哈希函数必须是本标准认可用于此用途的函数。

加密算法

本节为 V11.3 加密算法提供补充说明。

下表按优先顺序列出本标准认可的加密算法。

对称密钥算法	参考资料	状态
AES-256	FIPS 197	A
Salsa20	Salsa 20 specification	A
XChaCha20	XChaCha20 Draft	A
XSalsa20	Extending the Salsa20 nonce	A
ChaCha20	RFC 8439	A
AES-192	FIPS 197	A
AES-128	FIPS 197	L
2TDEA		D
TDEA (3DES/3DEA)		D
IDEA		D
RC4		D
Blowfish		D
ARC4		D

对称密钥算法	参考资料	状态
DES		D

AES 加密模式

AES等分组密码可采用不同的工作模式。电子密码本（ECB）等许多模式并不安全，严禁使用。伽罗瓦/计数器模式（GCM）和带 CBC-MAC 的计数器模式（CCM）可提供认证加密，应在现代应用中使用。

下表按优先顺序列出本标准认可的工作模式。

模式	是否认证	参考资料	状态	限制
GCM	是	NIST SP 800-38D	A	
CCM	是	NIST SP 800-38C	A	
CBC	否	NIST SP 800-38A	L	
CCM-8	是		D	
ECB	否		D	
CFB	否		D	
OFB	否		D	
CTR	否		D	

说明：

- 所有加密消息都必须经过认证。使用任何 CBC 模式时，都必须同时使用基于哈希的 MAC 算法验证消息。通常必须采用先加密后哈希（Encrypt-Then-Hash）的方式，但 TLS 1.2 使用先哈希后加密（Hash-Then-Encrypt）。如果无法满足这一要求，则不得使用 CBC。仅磁盘加密允许在不使用 MAC 算法的情况下进行加密。
- 使用 CBC 时，必须确保以恒定时间完成填充验证。
- 使用 CCM-8 时，MAC 标签仅提供 64 位安全强度，不符合要求 6.2.9 中至少 128 位安全强度的规定。
- 磁盘加密不在 ASVS 的范围内，因此本附录未列出本标准认可的磁盘加密方法。此类场景通常允许使用不带认证的加密，并普遍采用 XTS、XEX 和 LRW 模式。

密钥封装

密码学密钥封装（以及相应的密钥解封）是指使用额外的加密机制封装现有密钥，使原始密钥在传输等过程中不会直接暴露。用于保护原始密钥的附加密钥称为封装密钥。

当需要在不可信位置保护密钥，或通过不可信网络及应用内部传输敏感密钥时，可以使用此操作。不过，在决定执行封装或解封前，应认真考虑并理解原始密钥的属性，例如身份和用途。这项操作可能影响源系统、目标系统或应用的安全性，尤其会影响合规性，包括密钥功能（如签名）的审计记录以及适当的密钥存储方式。

密钥封装必须使用 AES-256，并遵循 NIST SP 800-38F，同时为未来的量子计算威胁做好准备。下表按优先顺序列出基于 AES 的加密模式：

密钥封装	参考资料	状态
KW	NIST SP 800-38F	A
KWP	NIST SP 800-38F	A

如果使用场景确有需要，可以使用 AES-192 和 AES-128，但必须在组织的密码学清单中记录使用理由。

认证加密

除磁盘加密外，加密数据必须使用某种认证加密（AE）方案防止未经授权的篡改，通常采用带关联数据的认证加密（AEAD）方案。

应用应优先使用本标准认可的 AEAD 方案，也可以将本标准认可的加密方案与本标准认可的 MAC 算法组合，并采用先加密后计算 MAC（Encrypt-then-MAC）的结构。

为兼容遗留应用，仍允许使用先计算 MAC 后加密（MAC-then-encrypt）。TLS v1.2 的旧密码套件采用了这种方式。

AEAD 机制	参考资料	状态
AES-GCM	SP 800-38D	A
AES-CCM	SP 800-38C	A
ChaCha-Poly1305	RFC 7539	A
AEGIS-256	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
AEGIS-128	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A

AEAD 机制	参考资料	状态
AEGIS-128L	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
Encrypt-then-MAC		A
MAC-then-encrypt		L

哈希函数

本节为 V11.4 哈希和基于哈希的函数提供补充说明。

通用场景下的哈希函数

下表列出了本标准认可用于数字签名等通用密码学场景的哈希函数：

- 本标准认可的哈希函数具有较强的抗碰撞能力，适用于高安全性应用。
- 其中部分算法配合适当的密码学密钥管理时具有较强的抗攻击能力，因此本标准也认可将其用于 HMAC、KDF 和 RBG。
- 输出长度不足 254 位的哈希函数无法提供足够的抗碰撞能力，不得用于数字签名或其他要求抗碰撞能力的场景。在其他场景中，这类函数只能用于遗留系统的兼容和验证，不得用于新的设计。

哈希函数	参考资料	状态	限制
SHA3-512	FIPS 202	A	
SHA-512	FIPS 180-4	A	
SHA3-384	FIPS 202	A	
SHA-384	FIPS 180-4	A	
SHA3-256	FIPS 202	A	
SHA-512/256	FIPS 180-4	A	
SHA-256	FIPS 180-4	A	
SHAKE256	FIPS 202	A	
BLAKE2s	BLAKE2: simpler, smaller, fast as MD5	A	
BLAKE2b	BLAKE2: simpler, smaller, fast as MD5	A	
BLAKE3	BLAKE3 one function, fast everywhere	A	

哈希函数		参考资料	状态	限制
SHA-224		FIPS 180-4	L	不适用于 HMAC、KDF、RBG 和数字签名
SHA-512/224		FIPS 180-4	L	不适用于 HMAC、KDF、RBG 和数字签名
SHA3-224		FIPS 202	L	不适用于 HMAC、KDF、RBG 和数字签名

哈希函数	参考资料	状态	限制
SHA-1	RFC 3174 & RFC 6194	L	不适用于 HMAC、KDF、RBG 和数字签名
CRC（任意长度）		D	
MD4	RFC 1320	D	
MD5	RFC 1321	D	

用于密码存储的哈希函数

为安全地对密码进行哈希处理，必须使用专用的哈希函数。这类慢速哈希算法会增加破解密码所需的计算量，从而降低暴力破解和字典攻击的风险。

KDF	参考资料	参数要求	状态
argon2id	RFC 9106	$t = 1: m \geq 47104$ (46 MiB), $p = 1$	A
		$t = 2: m \geq 19456$ (19 MiB), $p = 1$	A
		$t \geq 3: m \geq 12288$ (12 MiB), $p = 1$	A
scrypt	RFC 7914	$p = 1: N \geq 2^{17}$ (128 MiB), $r = 8$	A
		$p = 2: N \geq 2^{16}$ (64 MiB), $r = 8$	A
		$p \geq 3: N \geq 2^{15}$ (32 MiB), $r = 8$	A
bcrypt	一种面向未来、可调整的密码方案	$\text{cost} \geq 10$	A
PBKDF2-HMAC-SHA-512	NIST SP 800-132, FIPS 180-4	迭代次数 $\geq 210,000$	A

KDF	参考资料	参数要求	状态
PBKDF2- HMAC- SHA- 256	NIST SP 800-132, FIPS 180-4	迭代次数 $\geq 600,000$	A
PBKDF2- HMAC- SHA-1	NIST SP 800-132, FIPS 180-4	迭代次数 $\geq 1,300,000$	L

本标准认可的基于密码的密钥派生函数可用于存储密码。

密钥派生函数（KDF）

通用密钥派生函数

KDF	参考资料	状态
HKDF	RFC 5869	A
TLS 1.2 PRF	RFC 5248	L
基于 MD5 的 KDF	RFC 1321	D
基于 SHA-1 的 KDF	RFC 3174 & RFC 6194	D

基于密码的密钥派生函数

KDF	参考资料	参数要求	状态
argon2id	RFC 9106	$t = 1: m \geq 47104$ (46 MiB), $p = 1$	A
		$t = 2: m \geq 19456$ (19 MiB), $p = 1$	A
scrypt	RFC 7914	$p = 1: N \geq 2^{17}$ (128 MiB), $r = 8$	A
		$p = 2: N \geq 2^{16}$ (64 MiB), $r = 8$	A
		$p \geq 3: N \geq 2^{15}$ (32 MiB), $r = 8$	A

KDF	参考资料	参数要求	状态
PBKDF2- HMAC- SHA- 512	NIST SP 800-132, FIPS 180-4	迭代次数 $\geq 210,000$	A
PBKDF2- HMAC- SHA- 256	NIST SP 800-132, FIPS 180-4	迭代次数 $\geq 600,000$	A
PBKDF2- HMAC- SHA-1	NIST SP 800-132, FIPS 180-4	迭代次数 $\geq 1,300,000$	L

密钥交换机制

本节为 V11.6 公钥密码学提供补充说明。

密钥交换方案

所有密钥交换方案必须达到至少 112 位的安全强度，并按照下表选择实现参数。

方案	域参数	前向保密	状态
有限域 Diffie-Hellman (FFDH)	$L \geq 3072 \ \& \ N \geq 256$	是	A
椭圆曲线 Diffie-Hellman (ECDH)	$f \geq 256-383$	是	A
使用 RSA-PKCS#1 v1.5 的加密密钥传输		否	D

其中，各参数含义如下：

- k 表示 RSA 密钥的长度。
- 在有限域密码学中，L 表示公钥长度，N 表示私钥长度。
- f 表示 ECC 密钥长度的范围。

任何新实现都不得使用不符合 NIST SP 800-56A、NIST SP 800-56B 和 NIST SP 800-77 的方案。尤其不得在生产环境中使用 IKEv1。

Diffie-Hellman 群

本标准认可使用以下群实现 Diffie-Hellman 密钥交换。各群的安全强度记录在 NIST SP 800-56A 附录 D 和 NIST SP 800-57 第 1 部分修订版 5 中。

群	状态
P-224, secp224r1	A
P-256, secp256r1	A
P-384, secp384r1	A
P-521, secp521r1	A
K-233, sect233k1	A
K-283, sect283k1	A
K-409, sect409k1	A
K-571, sect571k1	A
B-233, sect233r1	A
B-283, sect283r1	A
B-409, sect409r1	A
B-571, sect571r1	A
Curve448	A
Curve25519	A
MODP-2048	A
MODP-3072	A
MODP-4096	A
MODP-6144	A
MODP-8192	A
ffdhe2048	A
ffdhe3072	A
ffdhe4096	A
ffdhe6144	A
ffdhe8192	A

消息认证码（MAC）

消息认证码（MAC）是一种用于验证消息完整性和真实性的密码学结构。MAC 以消息和秘密密钥作为输入，生成固定长度的标签，即 MAC 值。MAC 广泛用于 TLS/SSL 等安全通信协议，以确保通信双方交换的消息真实且未被篡改。

MAC 算法	参考资料	状态
HMAC-SHA-256	RFC 2104 & FIPS 198-1	A
HMAC-SHA-384	RFC 2104 & FIPS 198-1	A
HMAC-SHA-512	RFC 2104 & FIPS 198-1	A
KMAC128	NIST SP 800-185	A
KMAC256	NIST SP 800-185	A
BLAKE3 (keyed_hash 模式)	BLAKE3：一个可在各种环境中高效运行的函数	A
AES-CMAC	RFC 4493 & NIST SP 800-38B	A
AES-GMAC	NIST SP 800-38D	A
Poly1305-AES	The Poly1305-AES message-authentication code	A
HMAC-SHA-1	RFC 2104 & FIPS 198-1	L
HMAC-MD5	RFC 1321	D

数字签名

签名方案必须使用 NIST SP 800-57 第 1 部分认可的密钥长度和参数。

签名算法	参考资料	状态
EdDSA (Ed25519, Ed448)	RFC 8032	A
XEdDSA (Curve25519, Curve448)	XEdDSA	A

签名算法	参考资料	状态
ECDSA (P-256, P-384, P-521)	FIPS 186-4	A
RSA-RSSA-PSS	RFC 8017	A
RSA-SSA-PKCS#1 v1.5	RFC 8017	D
DSA（任意密钥长度）	FIPS 186-4	D

后量子加密标准

由于目前经过安全加固的代码和实现参考仍然很少，PQC实现必须符合 FIPS-203、FIPS-204 和 FIPS-205。另 见：<https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>

拟议的后量子混合 TLS 密钥协商方法 mlkem768x25519 已获得 Firefox 132 和 Chrome 131 等主流浏览器支持。该方法可用于密码学测试环境，也可在业界或政府批准的密码库提供支持时使用。

附录 D：建议

引言

在编制应用安全验证标准（ASVS）5.0版时，我们发现一些既有条目和新增建议不应作为 5.0 的要求纳入。这可能是因为按照 5.0 的定义，它们不属于 ASVS 的范围；也可能是因为它们虽然值得采纳，却不适合作为强制要求。

为保留其中仍有参考价值的内容，本附录收录了部分条目。

建议采用、且属于范围内的机制

以下条目属于 ASVS 的讨论范围。虽然不适合作为强制要求，但在构建安全应用时仍强烈建议予以考虑：

- 应提供密码强度计，帮助用户设置更强密码。
- 在应用根目录或.well-known 目录中创建公开可访问的 security.txt 文件，清楚定义供他人联系所有者报告安全问题的链接或电子邮件地址。
- 除可信服务层验证外，还应执行客户端输入验证，因为这有助于发现是否有人为攻击应用而绕过客户端控制。
- 使用 robots.txt 文件、X-Robots-Tag 响应头或 robots HTML meta 标签，防止意外可访问的敏感页面出现在搜索引擎中。
- 使用 GraphQL 时，在业务逻辑层实现授权逻辑，而不是在 GraphQL 或 resolver 层实现，以避免必须在每个独立接口上处理授权。

参考资料：

- security.txt 更多信息，包括 RFC 链接

软件安全原则

以下条目曾收录在 ASVS 中，但严格来说并不属于“要求”。它们是实施安全控制时可考虑的原则，遵循这些原则有助于提高控制措施的稳健性：

- 安全控制应集中、简单（设计经济性）、可验证其安全性，并且可复用。这应避免重复、缺失或无效控制。
- 尽可能使用已有且经过充分审查的安全控制实现，而不是依赖从零开始实现控制。
- 理想情况下，应使用单一访问控制机制访问受保护数据和资源。所有请求都应经过这一单一机制，以避免复制粘贴或不安全替代路径。
- 基于属性或功能的访问控制是一种推荐模式，即代码检查用户是否被授权访问某个功能或数据项，而不是只检查其角色。权限仍应通过角色分配。

软件安全流程

ASVS 5.0 删除了若干与安全流程有关的条目，但这些流程仍是值得采用的良好实践。有关如何有效落实这些流程，可以参考 OWASP SAMM 项目。此前收录在 ASVS 中的条目包括：

- 验证使用安全软件开发生命周期，在开发所有阶段处理安全问题。
- 验证在每次设计变更或迭代规划时使用威胁建模，以识别威胁、规划对策、促进适当的风险响应，并指导安全测试。
- 验证所有用户故事和功能都包含功能性安全约束，例如“作为用户，我应能够查看和编辑我的个人资料。我不应能够查看或编辑其他任何人的个人资料”。
- 验证所有开发人员和测试人员都可以访问安全编码检查清单、安全要求、指南或策略。
- 验证存在持续运行的流程，确保应用源代码不含后门、恶意代码（例如“切香肠”攻击、逻辑炸弹、时间炸弹），以及未记录或隐藏的功能（例如彩蛋、不安全的调试工具）。如果无法完整访问包括第三方库在内的所有源代码，就无法满足本条，因此它可能只适用于要求最高安全级别的应用。
- 验证已部署环境具备用于检测和响应配置漂移的机制。这可以包括使用不可变基础设施、根据安全基线自动重新部署，或使用漂移检测工具将当前状态与获批准的配置进行比较。
- 验证所有第三方产品、库、框架和服务都按其各自建议进行了配置加固。

参考资料：

- OWASP Threat Modeling Cheat Sheet
- OWASP Threat modeling
- OWASP Software Assurance Maturity Model Project
- Microsoft SDL

附录 E：贡献者

我们由衷感谢以下人员的贡献。他们自 ASVS 4.0.0 发布以来提交过评论或发起过拉取请求。

如果你发现任何错误，或希望以不同方式显示你的姓名，请告诉我们。

Johan Sydseter (sydseter)	luis servin (lfservin)	Oleksii Dovydkov (oleksiidov)	IZUKA Masahiro (maizuka)
James Sulinski (jsulinski)	Eli Saad (ThunderSon)	kkshitish9	Andrew van der Stock (vanderaj)
Rick M (kingthorin)	Bankde Eakasit (Bankde)	Michael Gargiullo (mgargiullo)	Raphael Dunant (Racater)
Cesar Kohl (cesarkohl)	inaz0	Joerg Bruenner (JoergBruenner)	David Deatherage (securitydave)
John Carroll (yosignals)	Jim Fenton (jimfenton)	Matteo Pace (M4tteoP)	Sebastien gioria (SPoint42)
Steven van der Baan (vdbaan)	Jeremy Bonghwan Choi (jeremychoi)	craig-shony	Riccardo Sirigu (ricsirigu)
Tomasz Wrobel (tw2as)	Alena Dubeshko (belalena)	Rafael Green (RafaelGreen1)	mjang-cobalt
clallier94	Kevin W. Wall (kwwall)	Jordan Sherman (jsherm-fwdsec / deleterepo)	Ingo Rauner (ingo-rauner)
Dirk Wetter (drwetter)	Moshe Zioni (moshe-apiiro)	Patrick Dwyer (coderpatros)	David Clarke (davidclarke-au)
Takaharu Ogasa (takaharuogasa)	Arkadii Yakovets (arkid15r)	Motoyasu Saburi (motoyasu-saburi)	leirn
wet-certitude	timhemel	RL Thornton (thornshadow99)	Thomas Bandt (aspnetde)
Roel Storms (roelstorms)	Jeroen Willemsen (commjoen)	anonymous-31	Kamran Saifullah (deFr0ggy)
Steve Springett (stevespringett)	Spyros (northdpole)	Hans Herrera (hansphp)	Marx314
CarlosAllendes	Yonah Russ (yruss972)	Sander Maijers (sanmai-NL)	Luboř Bretschneider (bretik)
Eva Sarafianou (esarafianou)	ataseren	Steve Thomas (Sc00bz)	Dominique RIGHETTO (righettod)
Steven van der Baan (svdb-ncc)	Michael Vacarella (Aif4thah)	Tonimir Kisasondi (tkisason)	Stefan Streichsbier (streichsbaer)
hi-unc1e	sb3k (starbuck3000)	mario-platt	Devdatta Akhawe (devd)

Michael Gissing (scolytus)	Jet Anderson (thatsjet)	Dave Wichers (davewichers)	Jonny Schnittger (JonnySchnittger)
Silvia Väli (silviavali)	jackgates73	1songb1rd	Timur - (timurozkul)
Gareth Heyes (hackvertor)	appills	suvikaartinen	chaals (chaals)
DanielPharos (AtlasHackert)	will Farrell (willfarrell)	Alina Vasiljeva (avasiljeva)	Paul McCann (ismisepaul)
Sage (SajjadPourali)	rbsec	Benedikt Bauer (mastacheata)	James Jardine (jamesjardine)
Mark Burnett (m8urnett)	dschwarz91	Cyber-AppSec (Cyber-AppSec)	Tib3rius
BitnessWise (bitnesswise)	damienbod (damienbod)	Jared Meit (jmeit-fwdsec)	Stefan Seelmann (sseelmann)
Brendan O' Connor (ussjoin)	Andrei Titov (andrettv)	Hans-Petter Fjeld (atluxity)	markehack
Neil Madden (NeilMadden)	Michael Geramb (mgeramb)	Osama Elnaggar (ossie-git)	mackowski
Ravi Balla (raviballa)	Hazana (hazanasec)	David Means (dmeans82)	Alexander Stein (tohch4)
BaeSenseii (baesenseii)	Vincent De Schutter (VincentDS)	S Bani (sbani)	Mitsuaki Akiyama (mak1yama)
Christopher Loessl (hashier)	victorxm	Michal Rada (michalradacz)	Veeresh Devireddy (drveresh)
MaknaSEO	darkzero2022	Liam (LiamDobbelaere)	Frank Denis (jedisct1)
Otto Sulin (ottosulin)	carllaw6885	Anders Johan Holmefjord (aholmis)	Richard Fritsch (rfritz)
mesutgungor	Scott Helme (ScottHelme)	Carlo Reggiani (carloreggiani)	Suyash Srivastava (suyash5053)
Mark Potter (markonweb)	Arjan Lamers (alamers)	Gøran Breivik (gobrtg)	flo-blg
Guillaume Déflache (guillaume-d)	Toufik Airane (toufik-airane)	Keith Hoodlet (securingdev)	Sinner (SoftwareSinner)
iloving	Jeroen Beckers (TheDauntless)	Joubin Jabbari (joubin)	yu fujioka (fujiokayu)

execjosh (execjosh)	Alicja Kario (tomato42)	Sidney Ribeiro (srjsoftware)	Gabriel Marquet (Gby56)
Drew Schulz (drschulz)	bedirhan	muralito	Ronnie Flathers (ropnop)
Philippe De Ryck (philippederyck)	Malte (mal33)	MazeOfThoughts	Andreas Falk (andifalk)
Javi (javixeneize)	Daniel Hahn (averell23)	borislav-c	Robin Wood (digininja)
miro2ns	Jan Dockx (jandockx)	vipinsaini434	priyanshukumar397
Nat Sakimura (sakimura)	Benjamin Häublein (BenjaminHae)	unknown-user-from	Ali Ramazan TAŞDELEN (alitasdln)
Pedro Escalera (oEscal)	Josh (josh-hemphill)	Tim Würtele (SEctim)	AviD (avidouglen)
SheHacksPurple (shehackspurple)	fcerullo-cycubix	Hector Eryx Paredes Camacho (heryxpc)	Irene Michlin (irene221b)
Jonah Y-M (TG-Techie)	Dhiraj Bahroos (bahroos)	Jef Meijvis (jefmeijvis)	IzmaDoesItbeta
Abdessamad TEMMAR (TmmmmmmR)	sectroyer	Soh Satoh (sohsatoh)	regoravalaz
james-t (james-bitherder)	Aram Hovsepyan (aramhovsepyan)	JaimeGomezGarciaSan	ValdiGit01
iwatachan (ishowta)	Vinod Anandan (VinodAnandan)	Kevin Kien (KevinKien)	paul-williamson-swoop
endergzzr	Radhwan Alshamamri (Rado0z)	Grant Ongers (rewtd)	Cure53 (cure53)
AliR2Linux	Ads Dawson (Gang-GreenTemperTatum)	William Reyor (BillReyor)	gabe (gcrow)
mascotter	luissaiz	Suren Manukyan (vx-sec)	Piotr Gliźniewicz (pglizniewicz)
Tadeusz Wachowski (tadeuszwachowski)	Nasir aka Nate (andesecc)	settantasette	Lars Haulin (LarsH)
Terence Eden (edent)	JasmineScholz	Arun Sivadasan (teavanist)	Yusuf GÜR (yusuffgur)
Troy Marshall (troymarshall)	Tanner Prynn (tprynn)	Nick K. (nickific)	raoul361

Azeem Ilyas (TheAxZim)	Evo Stamatov (avioli)	Tim Potter (timpotter87)	Gavin Ray (GavinRay97)
monis (demideus)	Marcin Hoppe (MarcinHoppe)	Grambulf (ramshazar)	Jordan Pike (computersarebad)
Jason Rogers (jason-invision)	Ben Hall (benbhall)	JamesPoppyCock (jamesly123)	WhiteHackLabs (whitehacklabs)
Alex Gaynor (alex)	Filip van Laenen (filipvanlaenen)	jeurgen	GraoMelo
Andreas Kurtz (ay-kay)	Tom Tervoort (TomTervoort)	old man (deveras)	Marco Schnüriger (marcortw)
stiiin	infoseclearn (teaminfoseclearn)	hljupkij	Noe (nmarher)
Lyz (lyz-code)	Martin Riedel (mrtnrdl)	KIM Jaesuck (tcaesvk)	Barbara Schachner (bschach)
René Reuter (AresSec)	carhackpils	Tyler (tyler2cr)	Hugo (hasousa)
Wouter Bloeyaert (Someniak)	Mark de Rijk (markderijkinfosec)	Ramin (picohub)	Philip D. Turner (philipdturner)
Will Chatham (willec)			